

الجزائريـــــــة الديمقراطيـــــة الشعبيـــة
وزارة التعليـــــم العـــــالي و البحـــــث العلـــمي
جامعة وهران للعلوم والتكنولوجيا محمد بوضياف
كلية الريـــاضيـــات و الإعـــلام الآلـــي
République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur Et de la Recherche Scientifique
Université des Sciences et de la Technologie d'Oran Mohamed BOUDIAF
Faculté des Mathématiques et Informatique
Département D'Informatique

Machine Learning Course

By Dr. Asmaa Ourdighi

Lecture 4

Chapitre 4: Classification

1. Perceptron simple (introduction aux réseaux)
2. Arbres de décision et forêts
3. k-plus proches voisins
4. Naïve Bayes
5. SVM (séparateurs linéaires)

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

Introduction

Qu'est-ce que classer ?

La classification est une tâche d'apprentissage supervisé. Le modèle reçoit des exemples décrits par des variables explicatives X et associés à une étiquette y. Il apprend une règle qui permettra d'attribuer une classe à une nouvelle observation.

Classer, c'est affecter à chaque observation x = (x_1,…,x_p) une étiquette y choisie parmi K classes : la variable à prédire est qualitative

Exemples :

→spam / non-spam ;

→malade / non-malade ;

→type de fleur ;

→signe de langue des signes ;

→client fidèle / client susceptible de partir.

Différence avec la régression. En régression y est un nombre ; ici y est une catégorie.

Binaire ou multiclasse. Binaire : $y \in \{-1, +1\}$. Multiclasse : $K > 2$.

Entrée X

Un vecteur de caractéristiques mesurées pour une observation.

Sortie y

Une catégorie discrète, par exemple 0 = refus et 1 = acceptation.

But

Apprendre à généraliser à des observations qui n'ont pas servi à l'apprentissage.

$$f : X \rightarrow \{C_1, C_2, \ldots, C_k\}$$

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

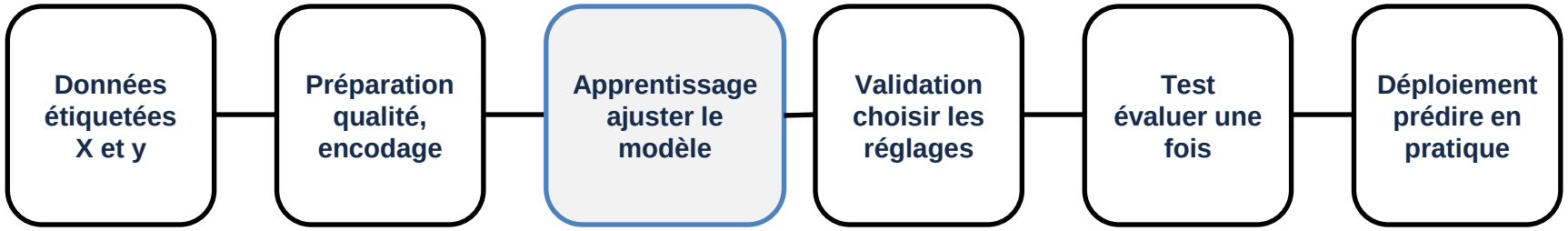
k-plus proches voisins

. . . . . .

Introduction

Qu'est-ce que classer ?

Du jeu de données à la décision



Séparation recommandée : apprentissage / validation / test

- **Apprentissage (≈ 60 %) :** les données sur lesquelles les **paramètres sont ajustés** (poids, coupures, voisins mémorisés).
- **Validation (≈ 20 %) :** sert à comparer les modèles et **à choisir les hyperparamètres** (profondeur, k, nombre d'arbres).
- **Test (≈ 20 %) :** utilisé une seule fois, à la fin, pour estimer la **performance en conditions réelles**.
- **Fuite de données :** toute information du test qui influence l'apprentissage (mise à l'échelle, sélection de variables) rend l'estimation optimiste.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

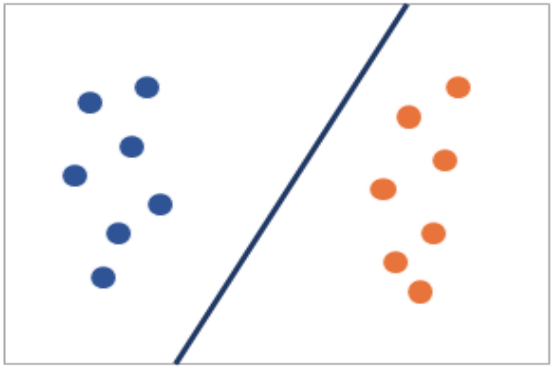
. . . . . .

Introduction

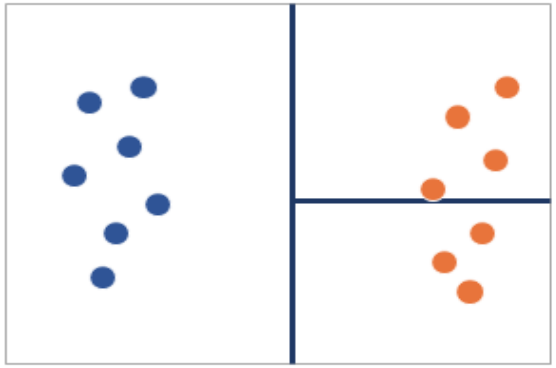
Qu'est-ce que classer ?

Classes, caractéristiques et frontière de décision

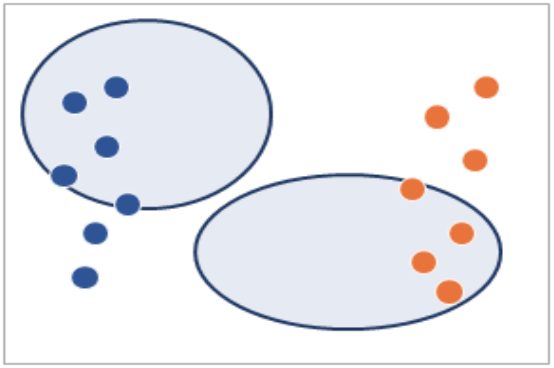
- Une observation est représentée par un vecteur $x = (x_1, x_2, \ldots, x_p)$, où p est le nombre de caractéristiques.
- Une classe y appartient à un ensemble fini : par exemple {0,1} en classification binaire ou {A,B,C} en classification multiclasse.
- Le modèle partitionne l'espace des caractéristiques en régions associées aux différentes classes.
- Une frontière de décision est la limite entre deux régions où la prédiction change de classe.



Linéaire (perceptron)



Rectangles (arbre)



Irrégulière (k-NN, k = 1)

Selon l'algorithme, la frontière est une droite/un hyperplan, une union de rectangles, ou une courbe très irrégulière.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

Introduction

Qu'est-ce que classer ?

Évaluer un classifieur : matrice de confusion

La matrice de confusion compare les classes réelles aux classes prédites. En binaire, on distingue :

	Prédit +	Prédit –
Réel +	VP = 40	FN = 10
Réel –	FP = 5	VN = 45

→ **TP (VP) :** positifs correctement prédits.
→ **TN (VN):** négatifs correctement prédits.
→ **FP :** négatifs **prédits à tort positifs**.
→ **FN :** positifs manqués par le modèle **prédits à tort positifs**.

Le coût de FP et FN dépend du problème : un faux négatif médical peut être plus préoccupant qu'un faux positif.

Calcul sur l'exemple (n = 100)

Accuracy = (40 + 45) / 100 = 0,85

Elle mesure la part de bonnes prédictions, toutes classes confondues : elle ne dit pas où se trouvent les erreurs.

Accuracy(Exactitude) = (VP + VN) / (VP + VN + FP + FN)

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

Introduction

Qu'est-ce que classer ?

Évaluer un classifieur : matrice de confusion

Précision = VP/(VP+FP)
Rappel = VP/(VP+FN)
F1 = 2PR/(P+R)

- **Précision : parmi les observations prédites positives, quelle part l'est vraiment** ? Importante quand les faux positifs coûtent cher (ex. bloquer un client honnête).
- **Rappel : parmi les vrais positifs, quelle part est retrouvée ?** Importante quand manquer un cas coûte cher (ex. dépistage).
- **F1-score : moyenne harmonique des deux** ; il est élevé seulement si précision et rappel le sont.

Compromis : abaisser le seuil de décision augmente le rappel mais diminue en général la précision.

Sur l'exemple VP=40, FP=5, FN=10

- Précision = 40/45 ≈ 0,89
- Rappel = 40/50 = 0,80
- F1 = 2×0,89×0,80/(0,89+0,80) ≈ 0,84

F1- score, pourquoi harmonique ?

Elle est tirée vers la plus petite valeur :
P = 1 et
R = 0,1 donnent
F1 ≈ 0,18, pas 0,55.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

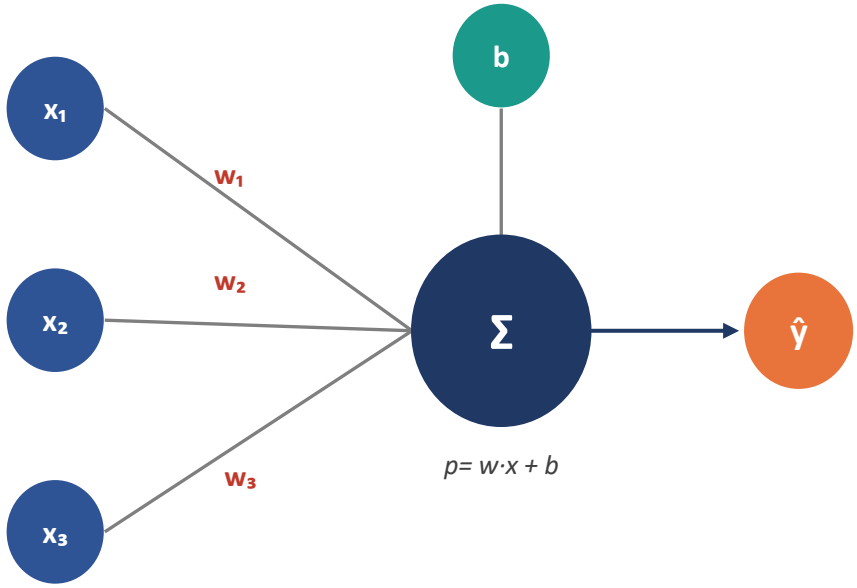
k-plus proches voisins

. . . . . .

Perceptron

Qu'est-ce qu'un perceptron ?

- **Inspiration.** Un neurone biologique reçoit des signaux, les additionne et s'active au-delà d'un seuil (McCulloch–Pitts 1943, Rosenblatt 1958).
- **Entrées et poids.** Chaque entrée x_j est multipliée par un poids w_j qui mesure son importance ; b est le biais (le seuil).
- **Étape 1, somme pondérée :** on calcule le score P.
- **Étape 2, activation :** on compare P à 0 pour décider la classe.



Chaque poids amplifie ou atténue son entrée ; le biais décale le seuil d'activation.

$$P = w_1 x_1 + \ldots + w_p x_p + b = w \cdot x + b$$

$\hat{y}$ = +1 si P ≥ 0, −1 sinon

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

Perceptron

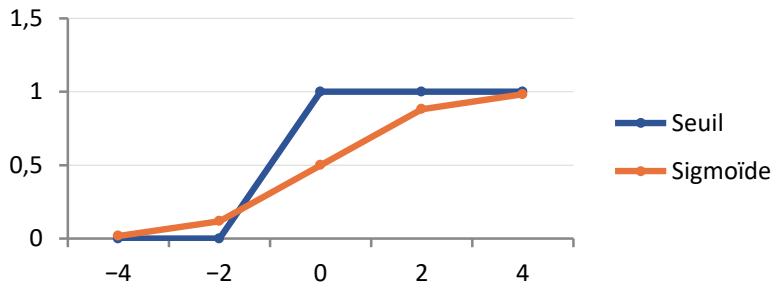
Équation fondamentale du perceptron

- **Score linéaire.** Chaque poids multiplie sa caractéristique ; la somme est décalée par le biais b.
- **Seuil.** Le score P est un nombre ; la prédiction ŷ est une étiquette obtenue en testant le signe de P.
- **Notation compacte.** En posant $x_0 = 1$ et $w_0 = b$, on écrit $P = w^T x$: le biais est un poids comme les autres.
- **Variantes.** Avec une sigmoïde à la place du seuil on obtient une probabilité $\sigma(P) \in]0,1[$ (régression logistique, neurone de réseau).

$$P = w^T x + b = \Sigma_{j=1}^{p} w_j x_j + b$$

$$\hat{y} = 1 \text{ si } P \geq 0 \text{ ; sinon } \hat{y} = 0$$

$$\sigma(P) = 1 / (1 + e^{-P})$$



Le score P est numérique, la prédiction ŷ est une étiquette ; le seuil zéro est absorbé par le biais.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

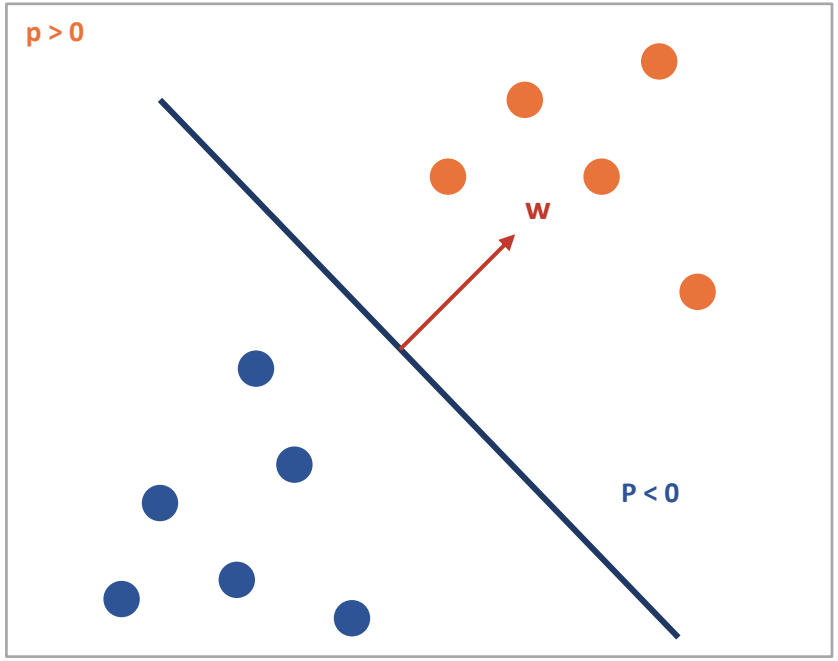
Perceptron

La frontière de décision du perceptron

Définition

L'ensemble des points où le score est nul : $w^T x + b = 0$.

- **Géométrie.** Droite en 2D, plan en 3D, hyperplan en dimension p. Le vecteur w lui est perpendiculaire et pointe vers la classe 1.
- **Distance.** Un point x est à la distance $|w^T x + b| / \|w\|$ de la frontière : plus $|p|$ est grand, plus la décision est « sûre ».
- **Limite fondamentale.** La frontière est linéaire : une seule droite pour séparer les classes.



$w_1x_1 + w_2x_2 + b = 0$

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

Perceptron

Comment le perceptron apprend-il ?

1. Initialiser w et b à zéro (ou à de petites valeurs).
2. Présenter un exemple x avec sa vraie classe y.
3. Calculer $\hat{y} = 1$ si $w^T x + b \geq 0$, sinon 0.
4. Bien classé ($y = \hat{y}$) : on ne change rien.
5. Mal classé : on corrige w et b dans le sens de y. Si $y = 1$ et $\hat{y} = 0$ on augmente le score ; si $y = 0$ et $\hat{y} = 1$ on le diminue.
6. Répéter sur plusieurs époques (parcours complets) jusqu'à zéro erreur ou un nombre maximal d'époques.

Le taux d'apprentissage η

Petit : corrections prudentes.
Grand : corrections brutales.
Pour le perceptron classique, η n'affecte pas la convergence si w est initialisé à 0.

Pourquoi ça marche ?

Corriger w de $\eta(y-\hat{y})x$ rapproche w de la direction de x (ou l'en éloigne), donc modifie le score de cet exemple dans le bon sens.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

Perceptron

Mise à jour des paramètres : mini-exemple

1. **Situation.** $x = (1, 2)$, vraie classe $y = 1$, taux $\eta = 0{,}1$, poids initiaux $w = (0{,}2 ; -0{,}3)$ et $b = 0$.
2. **Score avant :** $z = 0{,}2\times1 - 0{,}3\times2 + 0 = -0{,}4 < 0$, donc $\hat{y} = 0$: **erreur.**
3. **Erreur :** $y - \hat{y} = 1 - 0 = 1$.
4. **Poids :** $w = (0{,}2 ; -0{,}3) + 0{,}1\times1\times(1 ; 2) = \underline{(0{,}3 ; -0{,}1)}$.
5. **Biais :** $b = 0 + 0{,}1\times1 = \underline{0{,}1}$.
6. **Score après :** $p = 0{,}3\times1 - 0{,}1\times2 + 0{,}1 = 0{,}2 \geq 0$: l'exemple est maintenant bien classé ($\hat{y} = 1$).

Le score augmente de $\eta(\|x\|^2 + 1) = 0{,}1\times(5 + 1) = 0{,}6$: de $-0{,}4$ à $0{,}2$.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

Perceptron

Apprendre la porte ET : trace complète

Données.

- Entrées (0,0), (0,1), (1,0), (1,1) ;
- cibles 0, 0, 0, 1.
- Départ w = (0, 0), b = 0, η = 1.

Lecture. Chaque ligne donne les paramètres à la fin de l'époque et le nombre d'erreurs commises pendant celle-ci.

Résultat. Quand une époque ne produit aucune erreur, l'algorithme s'arrête : les données sont séparables.

Époque	Erreurs	Poids w	Biais b
1	2	(1 ; 1)	0
2	3	(2 ; 1)	-1
3	3	(2 ; 1)	-2
4	2	(2 ; 2)	-2
5	1	(2 ; 1)	-3
6	0	(2 ; 1)	-3

Frontière obtenue : $2x_1 + 1x_2 - 3 = 0$. Seul (1,1) est du côté positif : la porte ET est bien apprise.

Les solutions ne sont pas uniques : un autre ordre de présentation ou un autre point de départ donne d'autres (w, b).

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

Perceptron

Convergence et conditions

- **Cas séparable.** S'il existe un hyperplan séparant parfaitement les classes, le perceptron converge en un nombre fini de corrections.
- **Théorème de Novikoff (1962).** Si $\|x\| \leq R$ et si la marge de séparation est γ, le nombre de corrections est au plus $(R/\gamma)^2$. Une grande marge accélère l'apprentissage.
- **Cas non séparable.** Les corrections ne s'arrêtent jamais ; on fixe un nombre maximal d'époques et on garde les meilleurs paramètres.
- **Attention.** Converger ne veut pas dire généraliser au mieux : le perceptron s'arrête dès qu'il n'y a plus d'erreur, sans chercher la frontière la plus éloignée des points.

Si séparable

Nombre fini de mises à jour ; erreur d'apprentissage nulle.

Si non séparable

Oscillations ; arrêt par nombre d'époques ; évaluer sur validation.

Nombre de corrections ≤ $(R / \gamma)^2$ (R = rayon des données, γ = marge)

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

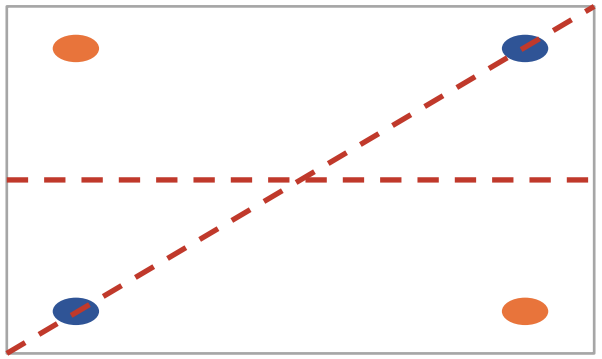
Perceptron

Limite centrale : le problème XOR

- **Définition.** XOR vaut 1 quand les deux entrées sont différentes, 0 quand elles sont identiques.
- **Géométrie.** Les deux points de classe 1 occupent des coins opposés, et ceux de classe 0 les deux autres : aucune droite ne les sépare.
- **Preuve rapide.** Il faudrait $b < 0$, $w_1+b \geq 0$, $w_2+b \geq 0$ et $w_1+w_2+b < 0$. Les deux inégalités du milieu donnent $w_1+w_2 \geq -2b > 0$, donc $w_1+w_2+b > -b > 0$: contradiction.
- **Conclusion.** Aucun nombre d'époques n'y change rien : c'est la forme linéaire de la frontière qui est en cause (Minsky et Papert, 1969).

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

Table de vérité et disposition des points



Pour apprendre XOR, il faut une représentation non linéaire ou plusieurs couches.

$\sigma(z) = 1 / (1 + e^{-z})$ $\quad$ $\mathrm{ReLU}(z) = \max(0, z)$

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

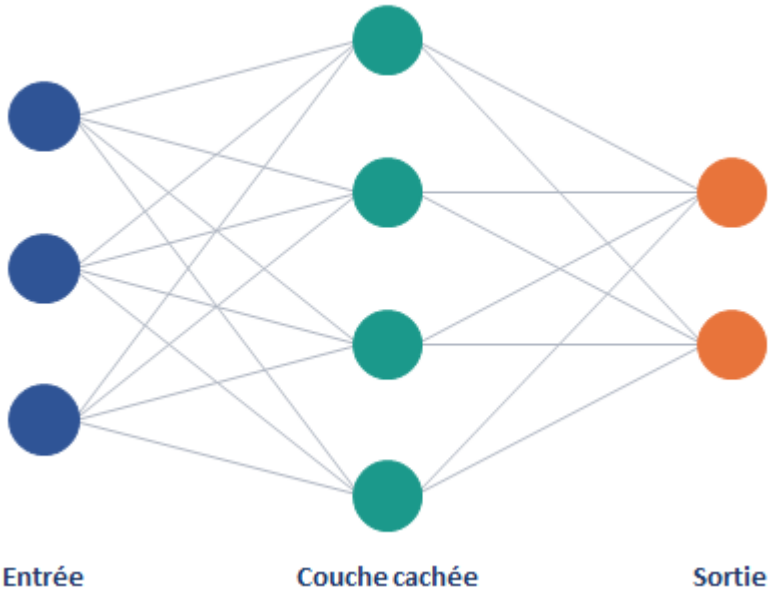
. . . . . .

Perceptron

Du perceptron au réseau de neurones

Non-linéarité + composition de couches ⇒ représentations plus expressives

- **MLP.** Des neurones en couches : entrée, une ou plusieurs couches cachées, sortie.
- **Non-linéarité.** Entre les couches, une activation (ReLU, sigmoïde, tanh) permet des frontières courbes.
- **XOR résolu.** Deux neurones cachés (« OU » et « ET ») suivis d'une sortie « OU et non ET » séparent XOR.
- **Sortie.** Sigmoïde pour 2 classes ; softmax pour K classes : $p_k = e^{z_k} / \Sigma_j e^{z_j}$
- **Apprentissage.** Par rétropropagation du gradient.



Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

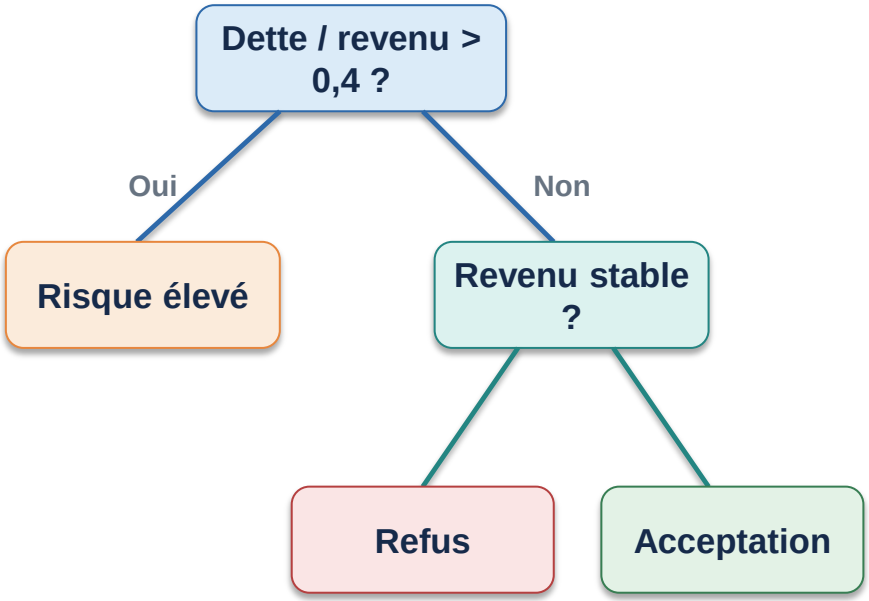
k-plus proches voisins

. . . . . .

Arbres de décision et forêts aléatoires

Anatomie d'un arbre de décision

- **Nœud interne :** un test sur une caractéristique (ex. dette/revenu > 0,4 ?).
- **Branche :** une réponse au test (oui / non).
- **Feuille :** la classe prédite, ou la distribution des classes des exemples qui y arrivent.
- **Racine :** le premier nœud, qui voit toutes les données.
- **Profondeur :** longueur du plus long chemin racine → feuille.
- **Prédire.** On descend de la racine à une feuille ; le chemin se lit comme une règle SI… ALORS…



Règle lue : SI dette/revenu ≤ 0,4 ET revenu stable ALORS Acceptation.
(Exemple fictif, sans valeur de conseil financier.)

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

Arbres de décision et forêts aléatoires

Comment l'arbre choisit-il une question ?

- **Candidats.** À chaque nœud, l'algorithme (CART) **examine des séparations possibles sur chaque caractéristique**.
- **Variable numérique :** coupure $x_j \leq t$. Les **seuils candidats** sont **les milieux entre valeurs consécutives triées**.
- **Variable catégorielle :** test d'une modalité ou d'un groupe de modalités, selon l'implémentation.
- **Évaluation. Chaque coupure partage les exemples en deux groupes** ; on mesure **leur pureté**.

  Chaque coupure divise l'ensemble des exemples du nœud en deux sous-groupes (enfants). On mesure ensuite **la pureté** de ces sous-groupes (c'est-à-dire à quel point les exemples d'un même groupe appartiennent à la même classe).

- **Choix.** On **garde la coupure qui réduit le plus l'impureté**, puis on recommence dans chaque enfant (récursion) jusqu'à **un critère d'arrêt**.

  L'algorithme retient la coupure qui **réduit le plus l'impureté** (ou maximise le gain d'information/la réduction d'impureté). Le processus est ensuite répété de manière récursive dans chaque sous-nœud jusqu'à atteindre un critère d'arrêt.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

Arbres de décision et forêts aléatoires

Comment l'arbre choisit-il une question ?

Critères d'arrêt

Pour éviter que l'arbre ne grandisse à l'infini et ne fasse du sur-apprentissage (*overfitting*), la récursion s'arrête lorsqu'une des conditions suivantes est remplie :

- **Nœud pur :** Tous les exemples du nœud appartiennent à la même classe.
- **Profondeur maximale :** L'arbre a atteint la limite de profondeur fixée par l'utilisateur.
- **Trop peu d'exemples :** Le nombre d'exemples restants dans le nœud est inférieur à un seuil minimal.
- **Gain trop faible :** La réduction d'impureté apportée par la meilleure coupure est insignifiante.

Algorithme glouton (Greedy Algorithm)

- **Principe :** L'algorithme de construction des arbres de décision est dit **glouton** parce qu'à chaque étape (chaque nœud), il choisit la coupure qui est **optimale localement** (celle qui maximise immédiatement la réduction d'impureté).
- **Conséquence :** Il n'y a **pas de garantie d'obtenir un arbre globalement optimal** à la fin, car un choix excellent à un niveau supérieur peut s'avérer sous-optimal pour la structure globale de l'arbre.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

Arbres de décision et forêts aléatoires

Comment l'arbre choisit-il une question ?

Critères d'arrêt

Pour éviter que l'arbre ne grandisse à l'infini et ne fasse du sur-apprentissage (*overfitting*), la récursion s'arrête lorsqu'une des conditions suivantes est remplie :

- **Nœud pur :** Tous les exemples du nœud appartiennent à la même classe.
- **Profondeur maximale :** L'arbre a atteint la limite de profondeur fixée par l'utilisateur.
- **Trop peu d'exemples :** Le nombre d'exemples restants dans le nœud est inférieur à un seuil minimal.
- **Gain trop faible :** La réduction d'impureté apportée par la meilleure coupure est insignifiante.

Algorithme glouton (Greedy Algorithm)

- **Principe :** L'algorithme de construction des arbres de décision est dit **glouton** parce qu'à chaque étape (chaque nœud), il choisit la coupure qui est **optimale localement** (celle qui maximise immédiatement la réduction d'impureté).
- **Conséquence :** Il n'y a **pas de garantie d'obtenir un arbre globalement optimal** à la fin, car un choix excellent à un niveau supérieur peut s'avérer sous-optimal pour la structure globale de l'arbre.

Arbres de décision et forêts aléatoires

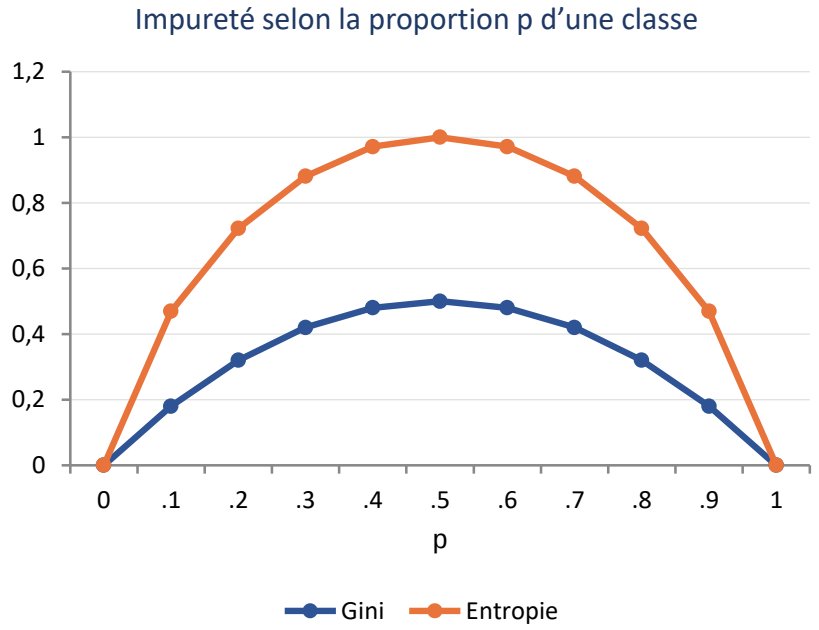
Mesurer l'impureté : Gini et entropie

- **Indice de Gini :** 0 pour un nœud pur, maximal quand les classes sont équilibrées (0,5 pour 2 classes).
- **Entropie :** 0 pour un nœud pur, maximale (1 bit à 2 classes) pour une répartition uniforme ; convention $0 \cdot \log_2 0 = 0$.
- p_k **:** proportion de la classe k dans le nœud.
- **En pratique** les deux critères donnent le plus souvent des coupures très proches.

$$\text{Gini}(S) = 1 - \Sigma_k \, p_k^2$$

$$H(S) = -\Sigma_k \, p_k \log_2 p_k$$

Exemple (7 A, 5 B) : Gini = 1 − (7/12)² − (5/12)² ≈ 0,486



Les deux courbes sont nulles aux extrémités (nœud pur) et maximales à p = 0,5.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

Arbres de décision et forêts aléatoires

Comment l'arbre choisit-il une question ?

Exemple : Prédire la réussite d'un étudiant

1. Présentation du problème

Une université souhaite construire **un arbre de décision capable de prédire si un étudiant va réussir ou échouer** à un examen.

On dispose d'un petit ensemble de données d'entraînement contenant 8 étudiants.

Les variables utilisées sont :

- Heures de révision : nombre d'heures consacrées à la préparation de l'examen.
- Résultat : réussite (Oui) ou échec (Non).

Étudiant	Heures de révision	Résultat
E1	1	Non
E2	2	Non
E3	3	Oui
E4	4	Non
E5	5	Oui
E6	6	Oui
E7	7	Non
E8	8	Oui

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

Arbres de décision et forêts aléatoires

Comment l'arbre choisit-il une question ?

Exemple : Prédire la réussite d'un étudiant

1. Présentation du problème

L'objectif est de construire un arbre qui permet de prédire le résultat d'un nouvel étudiant à partir de ses heures de révision.

2. Comment l'algorithme choisit-il la meilleure question ?

L'algorithme CART teste différentes questions sur les données.

Par exemple :

→ Question 1 : Heures de révision ≤ 2,5 ?

→ Question 2 : Heures de révision ≤ 4,5 ?

→ Question 3 : Heures de révision ≤ 6,5 ?

Pourquoi utiliser 2,5 plutôt que 2 ou 3 ?

Pour une variable numérique, CART teste généralement les seuils situés entre deux valeurs consécutives distinctes.

Par exemple, entre 2 et 3 : (2+3)/2=2,5

Chaque question divise les étudiants en deux groupes :

- **Groupe gauche : condition vérifiée**.
- Groupe droit : condition non vérifiée.

Étudiant	Heures de révision	Résultat
E1	1	Non
E2	2	Non
E3	3	**Oui**
E4	4	Non
E5	5	**Oui**
E6	6	**Oui**
E7	7	Non
E8	8	**Oui**

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

Arbres de décision et forêts aléatoires

Comment l'arbre choisit-il une question ?

Exemple : Prédire la réussite d'un étudiant

L'algorithme doit ensuite déterminer quelle question produit les **groupes les plus homogènes.**

Étudiant	Heures de révision	Résultat
E1	1	Non
E2	2	Non
E3	3	**Oui**
E4	4	Non
E5	5	**Oui**
E6	6	**Oui**
E7	7	Non
E8	8	**Oui**

3. Mesurer la pureté avec l'indice de Gini

L'indice de Gini mesure l'impureté d'un groupe.

Gini = $1 - \Sigma_k\, p_k^2$ (k=1 à K)

Dans notre problème, il existe deux classes : Oui et Non.

- p_{oui}: proportion d'étudiants ayant réussi.
- p_{non}: proportion d'étudiants ayant échoué.

Cas 1 : Un groupe parfaitement pur

Supposons qu'un groupe contienne 4 étudiants ayant tous réussi.

$Gini = 1 - (1^2 + 0^2) = 0$ Le groupe est parfaitement pur.

Cas 2 : Un groupe contenant les deux classes

Supposons qu'un groupe contienne 2 étudiants ayant réussi et 2 étudiants ayant échoué.

$Gini = 1 - [(2/4)^2 - (2/4)^2]$= 1-0,25-0,25= 0,5 **Le groupe est impur.**

À retenir : plus l'indice de Gini est proche de 0, plus le groupe est pur.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

Arbres de décision et forêts aléatoires

Comment l'arbre choisit-il une question ?

Exemple : Prédire la réussite d'un étudiant

4. Comparer deux questions candidates

Question A : Heures de révision ≤ 2,5 ?

Cette question sépare les données en deux groupes.

Calculons l'impureté de chaque groupe.

Groupe gauche :

$$Gini_G = 1 - (\frac{0}{2})^2 - (\frac{2}{2})^2 = 0$$

Groupe droit :

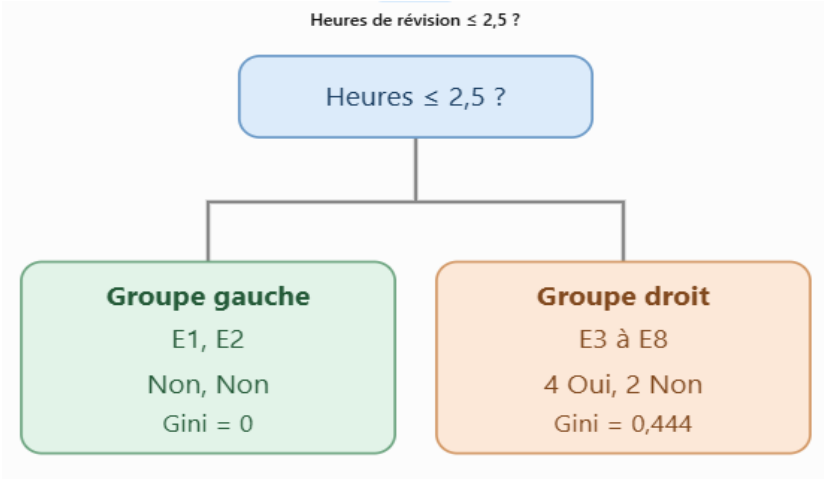
Il contient 4 Oui et 2 Non.

$$Gini_D = 1 - (\frac{4}{6})^2 - (\frac{2}{6})^2 = 0 \approx 0{,}444$$

L'impureté globale après la coupure est calculée comme un moyenne pondérée :

$$Gini_{split} = \frac{6}{8} Gini_D + \frac{2}{8} Gini_G \approx 0{,}3333$$

Étudiant	Heures de révision	Résultat
E1	1	Non
E2	2	Non
E3	3	**Oui**
E4	4	Non
E5	5	**Oui**
E6	6	**Oui**
E7	7	Non
E8	8	**Oui**



Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

Arbres de décision et forêts aléatoires

Comment l'arbre choisit-il une question ?

Exemple : Prédire la réussite d'un étudiant

Question B : Heures de révision ≤ 4,5 ?

Cette question donne les groupes suivants :

- Groupe gauche : E1, E2, E3, E4 → 1 Oui et 3 Non.
- Groupe droit : E5, E6, E7, E8 → 3 Oui et 1 Non.

Calculons l'impureté de chaque groupe.

Groupe gauche :

$$Gini_G = 1 - \left(\frac{1}{4}\right)^2 - \left(\frac{3}{4}\right)^2 = 0{,}375$$

Groupe droit : Il contient 3 Oui et 1 Non.

$$Gini_D = 1 - (\frac{3}{4})^2 - (\frac{1}{4})^2 = 0 = 0{,}375$$

L'impureté globale après la coupure est calculée comme une moyenne pondérée :

$$Gini_{split} = \frac{4}{8} Gini_D + \frac{4}{8} Gini_G = 0{,}375$$

Étudiant	Heures de révision	Résultat
E1	1	Non
E2	2	Non
E3	3	Oui
E4	4	Non
E5	5	Oui
E6	6	Oui
E7	7	Non
E8	8	Oui

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

Arbres de décision et forêts aléatoires

Comment l'arbre choisit-il une question ?

Exemple : Prédire la réussite d'un étudiant

Comparaison des questions

La question A est meilleure, car elle produit une impureté plus faible.

L'algorithme choisit donc : « Heures de révision ≤ 2,5 ? »

Il ne choisit pas nécessairement le seuil le plus intuitif, mais celui qui réduit le plus l'impureté.

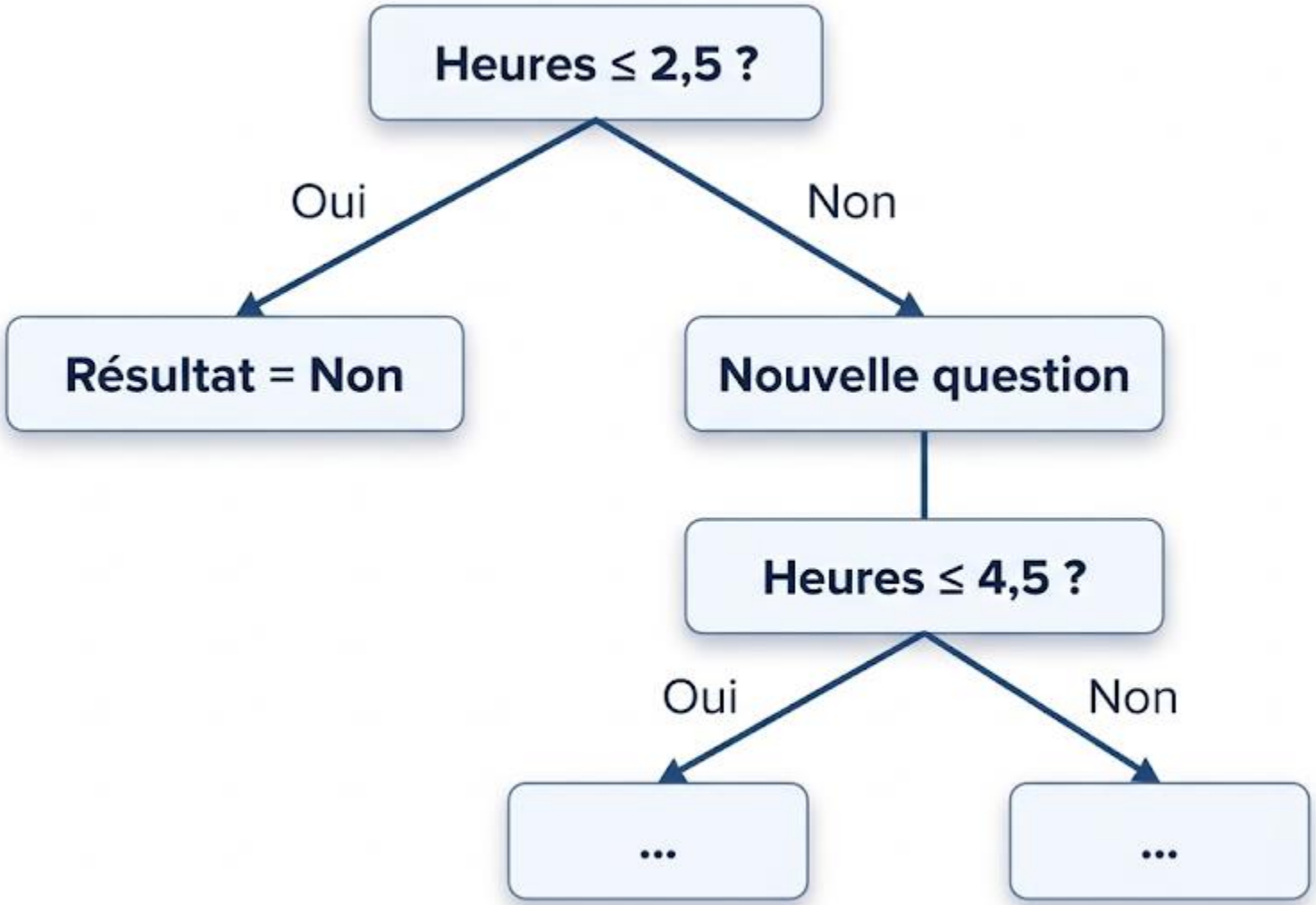
Question	Gini après la coupure
Heures ≤ 2,5	0,333
Heures ≤ 4,5	0,375

Étudiant	Heures de révision	Résultat
E1	1	Non
E2	2	Non
E3	3	**Oui**
E4	4	Non
E5	5	**Oui**
E6	6	**Oui**
E7	7	Non
E8	8	**Oui**

5. Construction récursive de l'arbre

Après avoir choisi la première question, CART recommence le même processus sur le groupe droit, qui contient encore des étudiants ayant réussi et des étudiants ayant échoué.

L'algorithme cherche une nouvelle question permettant de mieux séparer ces étudiants.


Heures ≤ 2,5 ?
Oui
Non
Résultat = Non
Nouvelle question
Heures ≤ 4,5 ?
Oui
Non
...
...

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

Arbres de décision et forêts aléatoires

Comment l'arbre choisit-il une question ?

Exemple : Prédire la réussite d'un étudiant

Le processus est récursif :

1) Choisir la meilleure question.
2) Diviser les données.
3) Examiner chaque groupe séparément.
4) Chercher une nouvelle question dans chaque groupe.
5) Continuer jusqu'à atteindre un critère d'arrêt.
6) Les critères d'arrêt peuvent être :
   - Le nœud est pur : tous les exemples appartiennent à la même classe.
   - La profondeur maximale de l'arbre est atteinte.
   - Le nombre d'exemples dans le nœud est trop faible.
   - Aucune coupure ne permet une réduction suffisante de l'impureté.

Étudiant	Heures de révision	Résultat
E1	1	Non
E2	2	Non
E3	3	**Oui**
E4	4	Non
E5	5	**Oui**
E6	6	**Oui**
E7	7	Non
E8	8	**Oui**

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

Arbres de décision et forêts aléatoires

Comment l'arbre choisit-il une question ?

Exemple : Prédire la réussite d'un étudiant

Étudiant	Heures de révision	Résultat
E1	1	Non
E2	2	Non
E3	3	**Oui**
E4	4	Non
E5	5	**Oui**
E6	6	**Oui**
E7	7	Non
E8	8	**Oui**

6. Pourquoi dit-on que l'algorithme est glouton (Greedy) ?

C'est le point essentiel à expliquer, le **meilleur choix local**

À chaque étape, l'algorithme choisit la question qui offre la meilleure réduction immédiate de l'impureté.

Il ne cherche pas à construire directement le meilleur arbre possible dans son ensemble.

Par exemple :

→ À la racine, il choisit le seuil 2,5.

→ Dans le groupe droit, il cherche la meilleure question locale.

→ Dans chacun des sous-groupes, il recommence de la même manière.

Il ne revient pas systématiquement sur ses décisions précédentes pour vérifier si une autre première question aurait permis d'obtenir un arbre plus petit.

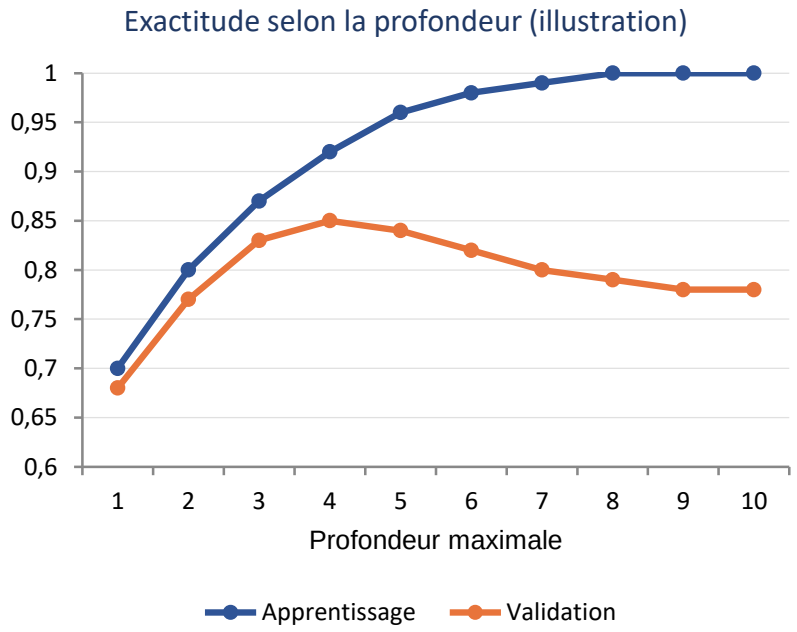
C'est pourquoi la construction est dite gloutonne.

Attention : un arbre qui sépare parfaitement les données d'entraînement n'est pas forcément le meilleur arbre pour les nouvelles données. Un arbre trop profond peut apprendre les particularités du jeu d'entraînement et présenter un risque de surapprentissage (overfitting).

Arbres de décision et forêts aléatoires

Pourquoi limiter la croissance d'un arbre ?

- **Arbre trop profond.** Il crée des feuilles minuscules, mémorise le bruit : excellente performance sur l'apprentissage, moins bonne sur de nouvelles données (surapprentissage).
- **Régularisation.** Limiter la profondeur, imposer un minimum d'exemples par feuille ou par coupure, exiger un gain minimal, ou élaguer.
- **Biais–variance.** Arbre trop simple : biais élevé. Arbre trop complexe : variance élevée. On cherche le compromis sur la validation.
- **Diagnostic.** Un écart qui se creuse entre exactitude d'apprentissage et de validation signale le surapprentissage.



Biais plus faible mais variance plus forte quand l'arbre devient trop complexe.

Arbres de décision et forêts aléatoires

Arbres de décision : forces et limites

- **Règles lisibles.** Un chemin racine → feuille s'explique à un non-spécialiste.
- **Peu de préparation.** Pas de standardisation nécessaire ; les relations non linéaires et les interactions sont captées.
- **Types mixtes.** Variables quantitatives et qualitatives selon l'implémentation.

Points de vigilance

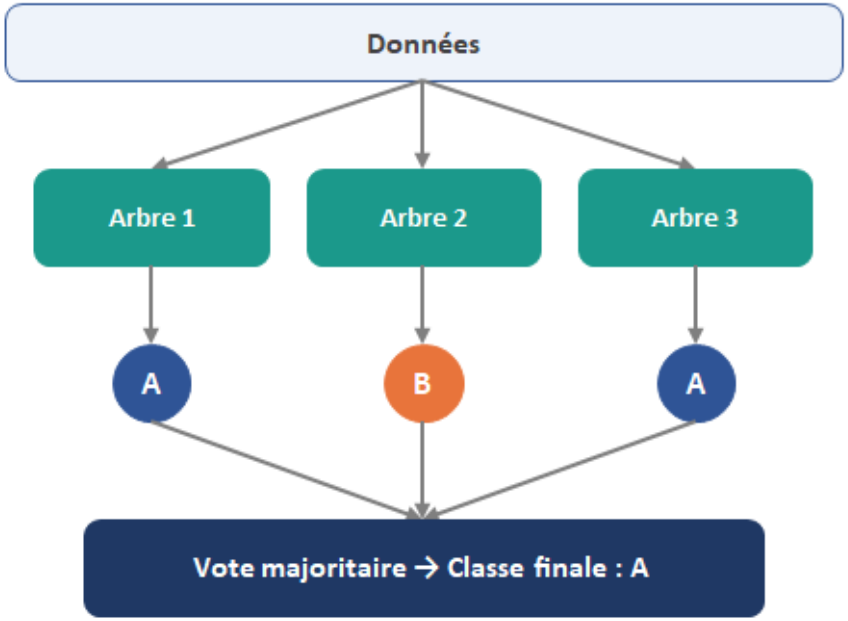
Instabilité : un petit changement des données peut modifier l'arbre.
Surapprentissage des arbres profonds.
Frontières « en escalier » peu adaptées aux frontières obliques.
Importance des variables parfois biaisée.

Un arbre unique privilégie l'interprétabilité ; une forêt, généralement, la robustesse prédictive.

Arbres de décision et forêts aléatoires

Forêt aléatoire : principe général

- **Définition.** Un ensemble d'arbres : chaque arbre apprend sur un échantillon bootstrap et, à chaque coupure, ne regarde qu'un sous-ensemble aléatoire de caractéristiques.
- **Deux sources d'aléa** rendent les arbres différents (décorrélés).
- **Agrégation.** Vote majoritaire en classification (moyenne en régression).
- **Idée centrale.** Agréger des modèles instables mais différents réduit la variance.



Moyenner des modèles différents stabilise la prédiction : $\mathrm{Var} \approx \rho\sigma^2 + (1-\rho)\sigma^2/B$ (ρ = corrélation entre arbres).

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

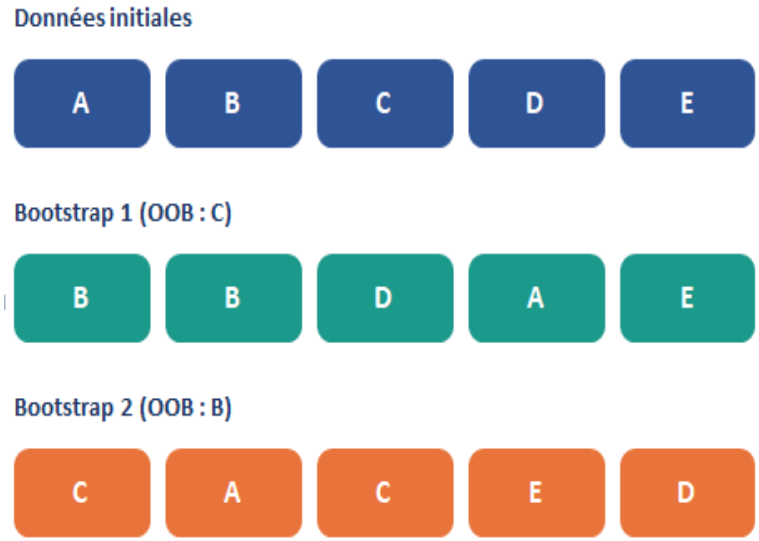
. . . . . .

Arbres de décision et forêts aléatoires

Forêt aléatoire

Bootstrap : créer des échantillons variés

- **Tirage avec remise.** On tire n observations parmi n ; certaines apparaissent plusieurs fois, d'autres jamais.
- **Un échantillon par arbre** diversifie la forêt.
- **Out-of-bag (OOB).** La probabilité qu'une observation soit absente d'un bootstrap est $(1 - 1/n)^n \rightarrow e^{-1} \approx 0{,}368$: environ 37 % des données sont OOB pour un arbre.
- **Usage.** Les OOB servent à estimer l'erreur **sans jeu de test séparé**.



Échantillons illustratifs, tirages avec remise.

Arbres de décision et forêts aléatoires

Forêt aléatoire

Pourquoi choisir des variables au hasard ?

Problème. Si une variable très informative domine, elle sera au sommet de tous les arbres : les arbres se ressemblent et l'agrégation apporte peu.

Solution. À chaque nœud, on tire m variables candidates parmi p ; le meilleur test est cherché parmi elles.

- **Effet.** La corrélation ρ entre arbres baisse, donc la variance de la moyenne aussi.
- **Réglage.** m est un hyperparamètre ; valeur usuelle : $m \approx \sqrt{p}$ en classification (p/3 en régression).

Exemple

p = 16 variables : chaque coupure n'en examine que m = 4, tirées au hasard.

Compromis

m petit : arbres très différents mais individuellement plus faibles.
m = p : on retrouve le bagging.

Diversité des arbres + agrégation ⇒ variance généralement réduite

Arbres de décision et forêts aléatoires

Forêt aléatoire

Comment la forêt prend-elle une décision ?

- **Vote.** Chaque arbre prédit une classe ; la forêt retient la plus fréquente.
- **Probabilités.** On peut aussi moyenner les probabilités estimées par les arbres (fréquences de classes dans les feuilles) et prendre la classe de score moyen maximal.
- **Exemple (B = 5).** Arbres : A, B, A, A, B → 3 votes A contre 2 B → classe A, probabilité estimée P(A) = 3/5 = 0,6.
- **Nombre d'arbres.** Plus il est grand, plus la prédiction est stable, mais le calcul et la mémoire augmentent ; au-delà d'un seuil, le gain devient faible.

Arbres 1 à 3

Arbre 1 : A • Arbre 2 : B • Arbre 3 : A

Résultat

2 votes A contre 1 vote B → $\hat{y}$ = A

$$\hat{y} = \text{mode}\{ h_1(x), h_2(x), \ldots, h_B(x) \}$$

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

Arbres de décision et forêts aléatoires

Forêt aléatoire

Comprendre le tirage bootstrap et les observations OOB dans une forêt aléatoire

Pour comprendre le fonctionnement du bootstrap et des observations Out-of-Bag (OOB), prenons un exemple concret de classification avec une forêt aléatoire (Random Forest).

L'objectif est de comprendre :

→ Pourquoi chaque arbre **apprend sur un échantillon différent**.

→ Comment certaines **observations peuvent apparaître plusieurs fois** dans un même échantillon.

→ Pourquoi environ **37 % des observations** ne sont pas utilisées pour entraîner un arbre donné.

→ Comment **ces observations permettent d'estimer l'erreur de la forêt sans utiliser un jeu de test séparé**.

Étudiant	Heures de révision	Résultat
E1	1 h	Échec
E2	2 h	Échec
E3	3 h	Échec
E4	4 h	Réussite
E5	5 h	Réussite
E6	6 h	Réussite
E7	7 h	Réussite
E8	8 h	Réussite
E9	9 h	Réussite
E10	10 h	Réussite

Exemple : prédire si un étudiant réussira un examen

Supposons que nous disposions d'un petit jeu de données de 10 étudiants.

Nous souhaitons construire une forêt aléatoire qui prédit si un étudiant réussira son examen en fonction de ses heures de révision.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

Arbres de décision et forêts aléatoires

Forêt aléatoire

Comprendre le tirage bootstrap et les observations OOB dans une forêt aléatoire

Pourquoi parle-t-on de tirage avec remise ?

Parce que E2 peut être tiré une première fois, remis dans la population, puis tiré une deuxième fois.

Lors de l'entraînement, les deux occurrences de E2 sont considérées comme deux lignes présentes dans l'échantillon bootstrap. Elles contribuent donc deux fois à l'apprentissage de l'arbre.

3. Pourquoi chaque arbre apprend-il sur un échantillon différent ?

Construisons maintenant les échantillons bootstrap de trois arbres.

Étudiant	Heures de révision	Résultat
E1	1 h	Échec
E2	2 h	Échec
E3	3 h	Échec
E4	4 h	Réussite
E5	5 h	Réussite
E6	6 h	Réussite
E7	7 h	Réussite
E8	8 h	Réussite
E9	9 h	Réussite
E10	10 h	Réussite

Arbre 1

E1, E2, E2, E4, E5, E6, E7, E7, E9, E10

Observations absentes : E3, E8

Arbre 2

E1, E1, E3, E4, E4, E6, E8, E9, E9, E10

Observations absentes : E2, E5, E7

Arbre 3

E2, E3, E3, E4, E5, E5, E7, E8, E10, E10

Observations absentes : E1, E6, E9

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

Arbres de décision et forêts aléatoires

Forêt aléatoire

Comprendre le tirage bootstrap et les observations OOB dans une forêt aléatoire

Nous remarquons que :

- Les trois arbres sont construits à partir de la même population initiale.
- Chaque arbre reçoit un échantillon bootstrap différent.
- Les observations répétées influencent davantage l'apprentissage de l'arbre.
- Les observations absentes ne participent pas à l'entraînement de cet arbre.

Cette diversité contribue à rendre les arbres différents les uns des autres.

Dans une forêt aléatoire, la diversité provient également de la sélection aléatoire des caractéristiques à chaque division d'un arbre.

Étudiant	Heures de révision	Résultat
E1	1 h	Échec
E2	2 h	Échec
E3	3 h	Échec
E4	4 h	Réussite
E5	5 h	Réussite
E6	6 h	Réussite
E7	7 h	Réussite
E8	8 h	Réussite
E9	9 h	Réussite
E10	10 h	Réussite

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

Arbres de décision et forêts aléatoires

Forêt aléatoire

Comprendre le tirage bootstrap et les observations OOB dans une forêt aléatoire

4. Comprendre les observations Out-of-Bag (OOB)

Les observations OOB sont les observations qui n'ont pas été sélectionnées dans l'échantillon bootstrap d'un arbre donné.

Reprenons notre exemple.

- Pour l'Arbre 1 : Les étudiants E1, E2, E4, E5, E6, E7, E9 et E10 sont utilisés pour entraîner l'arbre.

Les étudiants E3 et E8 ne sont pas utilisés pour entraîner cet arbre.

Ainsi :

$OOB_1 = \{\text{E3}, E8\}$

Étudiant	Heures de révision	Résultat
E1	1 h	Échec
E2	2 h	Échec
E3	3 h	Échec
E4	4 h	Réussite
E5	5 h	Réussite
E6	6 h	Réussite
E7	7 h	Réussite
E8	8 h	Réussite
E9	9 h	Réussite
E10	10 h	Réussite

Mais attention : E3 et E8 ne sont pas supprimés du jeu de données initial.

Ils sont simplement absents de l'entraînement de l'Arbre 1.

On peut donc utiliser cet arbre pour prédire le résultat de E3 et E8, alors que leurs résultats réels n'ont pas été utilisés pour entraîner cet arbre.

C'est précisément l'intérêt des observations OOB.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

Arbres de décision et forêts aléatoires

Forêt aléatoire

Comprendre le tirage bootstrap et les observations OOB dans une forêt aléatoire

A. Pourquoi environ 37 % des observations sont-elles OOB ?

Considérons une observation particulière, par exemple E3.

Lors d'un tirage, la probabilité de sélectionner E3 est : $P(sélectionner\ E3) = \frac{1}{n}$

La probabilité de ne pas sélectionner E3 lors d'un tirage est donc : $P(absence) = 1 - \frac{1}{n}$

Or, nous effectuons n tirages avec remise.

La probabilité que E3 ne soit sélectionné lors d'aucun des n tirages est : $P(E3\ absence) = (1 - \frac{1}{n})^{n}$

Lorsque n devient suffisamment grand :

$\lim_{n\to+\infty} \left(1 - \frac{1}{n}\right)^{n} = e^{-1}$

Et puisque : $e^{-1} \approx 0{,}368$

Nous obtenons : $P(OOB) \approx 36{,}8\%$

Cela signifie qu'en moyenne, environ 37 % des observations initiales sont absentes de l'échantillon bootstrap d'un arbre.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

Arbres de décision et forêts aléatoires

Forêt aléatoire

Comprendre le tirage bootstrap et les observations OOB dans une forêt aléatoire

B. Vérification avec notre exemple de 10 étudiants

Pour (n=10) :

$$P(OOB) = (1 - \frac{1}{10})^{10} = (0{,}9)^{10} \approx 0{,}3487$$

Soit environ 34,87 %.

Pour notre petit jeu de données, cela correspond en moyenne à : $10 \times 0{,}3487 \approx 3{,}49$

Donc, environ **3 à 4 étudiants seront absents de l'échantillon bootstrap d'un arbre**.

Dans notre exemple, l'Arbre 1 possède seulement deux observations OOB. Ce n'est pas contradictoire : le résultat de 3,49 est **une moyenne théorique**, et non une valeur imposée pour chaque arbre.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

Arbres de décision et forêts aléatoires

Forêt aléatoire

Comprendre le tirage bootstrap et les observations OOB dans une forêt aléatoire

5. Comment utiliser les observations OOB pour estimer l'erreur ?

C'est la partie la plus importante.

Supposons maintenant que nous construisions une forêt de 100 arbres.

Chaque arbre possède son propre échantillon bootstrap.

Pour un étudiant donné, par exemple E5 :

→ Certains arbres utilisent E5 pour leur apprentissage.

→ D'autres arbres ne sélectionnent jamais E5.

Ces derniers arbres sont appelés les arbres OOB de E5.

Nous allons utiliser uniquement les arbres OOB de E5 pour prédire son résultat.

Exemple avec l'étudiant E5

Supposons que, parmi les 100 arbres :

- 65 arbres ont utilisé E5 pendant l'entraînement.
- 35 arbres n'ont pas utilisé E5.

Les 35 arbres OOB peuvent donc prédire le résultat de E5.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

Arbres de décision et forêts aléatoires

Forêt aléatoire

Comprendre le tirage bootstrap et les observations OOB dans une forêt aléatoire

5. Comment utiliser les observations OOB pour estimer l'erreur ?

Imaginons que leurs prédictions soient :
Le résultat majoritaire est :

$$\widehat{y_{E5}} = Réussite$$

Or, le résultat réel de E5 est également Réussite.
La prédiction OOB de cet étudiant est donc correcte.

Prédiction OOB	Nombre de votes
Réussite	30
Échec	5
Total	35

Pourquoi cette prédiction est-elle intéressante ?

Parce que E5 n'a jamais été utilisé pour entraîner les 35 arbres qui ont produit cette prédiction.
La prédiction OOB constitue donc une estimation de la capacité de généralisation de la forêt, sans entraîner un modèle supplémentaire sur un jeu de validation séparé.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)

Arbres de décision et forêts

k-plus proches voisins

. . . . . .

Arbres de décision et forêts aléatoires

Forêt aléatoire

Comprendre le tirage bootstrap et les observations OOB dans une forêt aléatoire

6. Calcul de l'erreur OOB de la forêt

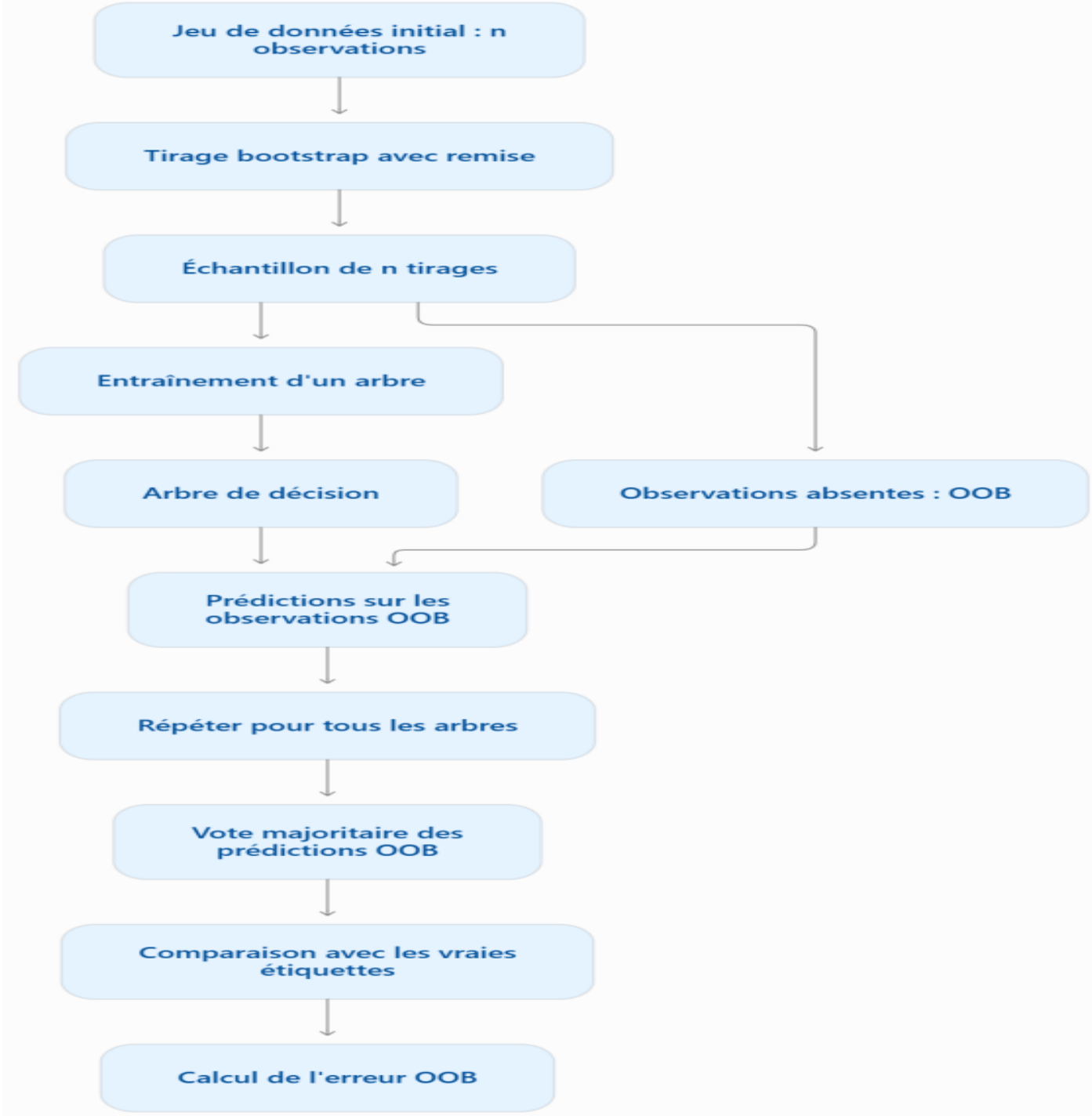
Nous répétons le même processus pour chacun des 10 étudiants.
Pour chaque étudiant :

→ Nous identifions les arbres pour lesquels il est OOB.
→ Nous récupérons leurs prédictions.
→ Nous effectuons un vote majoritaire.
→ Nous comparons la prédiction obtenue avec le résultat réel.

Prenons un exemple simplifié
Supposons que les 10 étudiants disposent chacun d'une prédiction OOB valide.
Nous avons :
9 prédictions correctes.
1 prédiction incorrecte.
L'erreur OOB est :

$$Erreur\ OOB = \frac{Nombre\ de\ prédictions\ incorrectes}{Nombre\ total\ de\ prédictions\ OOB} = \frac{1}{10} = 0{,}1$$

Étudiant	Résultat réel	Prédiction OOB	Correct ?
E1	Échec	Échec	Oui
E2	Échec	Échec	Oui
E3	Échec	Échec	Oui
E4	Réussite	Réussite	Oui
E5	Réussite	Échec	Non
E6	Réussite	Réussite	Oui
E7	Réussite	Réussite	Oui
E8	Réussite	Réussite	Oui
E9	Réussite	Réussite	Oui
E10	Réussite	Réussite	Oui


Jeu de données initial : n observations
Tirage bootstrap avec remise
Échantillon de n tirages
Entraînement d'un arbre
Arbre de décision
Observations absentes : OOB
Prédictions sur les observations OOB
Répéter pour tous les arbres
Vote majoritaire des prédictions OOB
Comparaison avec les vraies étiquettes
Calcul de l'erreur OOB

Arbres de décision et forêts aléatoires

Forêt aléatoire

Arbre unique ou forêt aléatoire ?

- **Choisir selon** la validation, les contraintes opérationnelles et le besoin d'explicabilité.
- **Règle pratique.** Besoin de transparence totale → arbre peu profond. Objectif de performance sur données tabulaires → forêt.

Arbre de décision

Compact, visualisable ; une décision s'explique par un chemin. Instable, tend à surapprendre.

Forêt aléatoire

Plus robuste, capte interactions et non-linéarités. Plus difficile à expliquer globalement ; coût mémoire et calcul supérieur.

k plus proches voisins (k-NN)

Définition

L'intuition : « dis-moi qui sont tes voisins… »

Le k-plus proches voisins (k-Nearest Neighbors, k-NN) est une méthode supervisée non paramétrique. Pour prédire la classe d'un nouvel individu, elle identifie les k observations d'apprentissage les plus proches selon une mesure de distance, puis agrège leurs étiquettes, souvent par vote majoritaire.

INTUITION

« Les objets qui se ressemblent (qui sont proches dans l'espace des caractéristiques) ont de fortes chances d'appartenir à la même classe. »

Deux particularités le distinguent de la plupart des autres méthodes :

Il n'y a pas de phase d'apprentissage au sens strict :

→ **Il n'y a pas de phase d'apprentissage au sens strict** l'algorithme se contente de mémoriser les données. On parle d'apprentissage *paresseux* (*lazy learning*) ou *basé sur les instances*.

→ **Il ne suppose aucune forme particulière** pour la frontière entre les classes : il est dit *non paramétrique*. Il peut donc s'adapter à des frontières très irrégulières.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
. . . . . .

k plus proches voisins (k-NN)

Définition

principe de l'algorithme

Soit un jeu d'apprentissage de n exemples (x_1, y_1), …, (x_n, y_n), où chaque x_i est décrit par p caractéristiques et y_i est sa classe. Pour classer un nouvel exemple x :

1. **Choisir** la valeur de k et une distance d.
2. **Calculer** la distance d(x, x_i) entre x et chacun des n exemples d'apprentissage.
3. **Trier** ces distances par ordre croissant et **retenir** les k exemples les plus proches : ils forment l'ensemble $N_{k(x)}$, le voisinage de x.
4. **Voter** : la classe prédite est celle qui apparaît le plus souvent dans $N_{k(x)}$.
5. Mathématiquement, la règle de décision s'écrit :
6. où $\mathbf{1}(y_i = c)$vaut 1 si le voisin i appartient à la classe c et 0 sinon : la somme compte donc, pour chaque classe c, le nombre de voisins qui en font partie, et argmax désigne la classe qui obtient le plus de voix.

Algorithme 1 — k-NN en pseudo-code

Entrées : jeu d'apprentissage (X, y), nouvel exemple x, entier k, distance d
1. pour chaque exemple x_i de X :
2. calculer dist_i = d(x, x_i)
3. trier les exemples par dist_i croissante
4. V = les k premiers exemples de cette liste
   # les k voisins
5. compter, pour chaque classe c, le nombre de voisins de V dans c
6. renvoyer la classe c ayant le plus grand compte

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
. . . . . .

k plus proches voisins (k-NN)

Définition

Mesurer la proximité : les distances

Tout repose sur la notion de « proche ». Il faut donc une **distance** : une fonction d(x, y) qui renvoie un nombre ≥ 0, valant 0 uniquement si x = y, et qui est d'autant plus grande que x et y sont dissemblables. Voici les plus courantes pour des variables numériques $x = (x_1,\ldots,x_p)$ et $y = (y_1,\ldots,y_p)$.

La distance euclidienne
C'est la distance « à vol d'oiseau », celle que l'on mesure avec une règle. Elle généralise le théorème de Pythagore :

$$d(x,y) = \sqrt{\sum_{j=1}^{p} (x_j - y_j)^2}$$

La distance de Manhattan
Elle additionne les écarts sur chaque axe, comme un taxi qui doit suivre les rues d'un quadrillage (d'où le nom, inspiré du plan de Manhattan) :

$$d_1(x,y) = \sum_{j=1}^{p} |x_j - y_j|$$

La distance de Tchebychev et la distance de Minkowski
La distance de Tchebychev ne retient que le plus grand écart sur un axe. La distance de Minkowski est une famille qui contient les précédentes : q = 1 donne Manhattan, q = 2 donne l'euclidienne, et q → ∞ donne Tchebychev.

$$d_\infty(x,y) = \max_j |x_j - y_j|$$

$$d_q(x,y) = \left(\sum_{j=1}^{p} |x_j - y_j|^q \right)^{1/q}$$

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
. . . . . .

k plus proches voisins (k-NN)

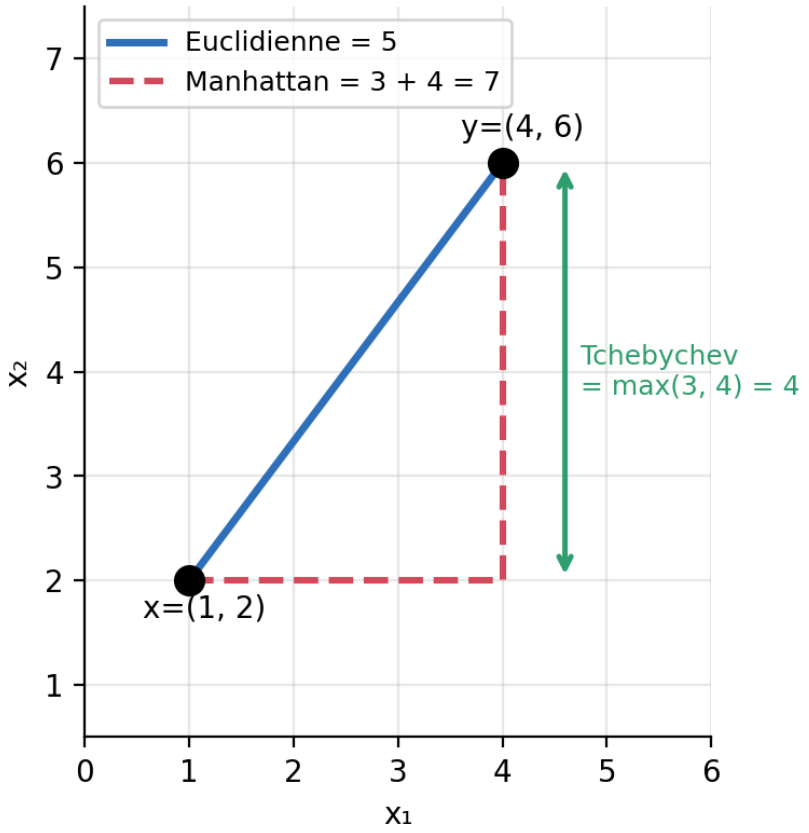
Définition

Mesurer la proximité : les distances

EXEMPLE

Soient x = (1, 2) et y = (4, 6). Les écarts sont $|1 - 4| = 3$ et $|2 - 6| = 4$.

- Euclidienne : $\sqrt{(3^2 + 4^2)} = \sqrt{25} = \mathbf{5}$
- Manhattan : $3 + 4 = \mathbf{7}$
- Tchebychev : $\max(3, 4) = \mathbf{4}$



Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
. . . . . .

k plus proches voisins (k-NN)

Définition

Variables catégorielles : la distance de Hamming

Variables catégorielles : la distance de Hamming

Si les caractéristiques sont des catégories (couleur, pays, oui/non), on compte simplement le nombre de caractéristiques qui diffèrent :

$$d_H(x, y) = \sum_{j=1}^{p} \mathbf{1}(x_j \neq y_j)$$

EXEMPLE

x = (rouge, petit, rond) et y = (rouge, grand, rond) : une seule caractéristique diffère, donc d_H(x, y) = **1**.

Distance	Quand l'utiliser ?	Remarque
Euclidienne	Choix par défaut, variables numériques continues	Sensible aux grandes différences (carrés)
Manhattan	Nombreuses variables, données avec des valeurs aberrantes	Plus robuste que l'euclidienne
Tchebychev	Quand seul le pire écart compte	Rare en pratique
Hamming	Variables catégorielles ou binaires	Compte les différences

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
. . . . . .

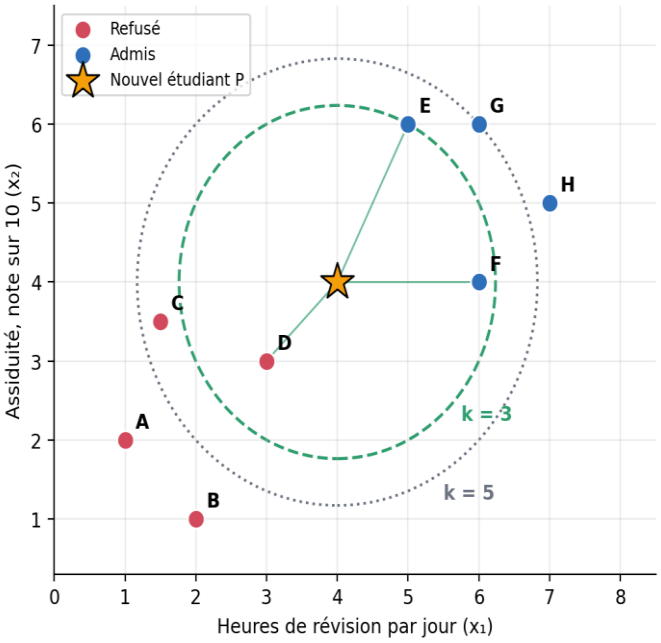
k plus proches voisins (k-NN)

Exemple complet, pas à pas

Une enseignante veut prédire si un étudiant sera **Admis** ou **Refusé** à un examen. Elle a relevé, pour huit anciens étudiants, le nombre d'heures de révision par jour (x_1) et leur assiduité en cours (x_2, note sur 10). Un nouvel étudiant **P** révise 4 heures par jour avec une assiduité de 4/10 : sera-t-il admis ?

Table Données d'apprentissage et distances euclidiennes à P = (4, 4)

Étudiant	x_1 (heures)	x_2 (assiduité)	Classe	Distance à P	Rang
A	1	2	Refusé	$\sqrt{(9 + 4)} = 3,61$	7 (ex æquo)
B	2	1	Refusé	$\sqrt{(4 + 9)} = 3,61$	7 (ex æquo)
C	1,5	3,5	Refusé	$\sqrt{(6,25 + 0,25)} = 2,55$	4
D	3	3	Refusé	$\sqrt{(1 + 1)} = 1,41$	**1**
E	5	6	Admis	$\sqrt{(1 + 4)} = 2,24$	**3**
F	6	4	Admis	$\sqrt{(4 + 0)} = 2,00$	**2**
G	6	6	Admis	$\sqrt{(4 + 4)} = 2,83$	**5**
H	7	5	Admis	$\sqrt{(9 + 1)} = 3,16$	6



k plus proches voisins (k-NN)

Exemple complet, pas à pas

Classons les voisins par distance croissante :

1. **D** (Refusé, 1,41),
2. **F** (Admis, 2,00),
3. **E** (Admis, 2,24),
4. **C** (Refusé, 2,55),
5. **G** (Admis, 2,83), …

k	Voisins retenus	Vote	Prédiction
1	D	1 **Refusé**	Refusé
3	D, F, E	1 Refusé, **2 Admis**	**Admis**
5	D, F, E, C, G	2 Refusé, **3 Admis**	**Admis**

À RETENIR

Avec k = 1, P est classé « Refusé » parce que un seul voisin, D, est un peu plus proche. Dès k = 3, la majorité des voisins plaide pour « Admis ». **Un k trop petit rend donc la prédiction fragile : elle dépend d'un seul point, peut-être atypique.**

Variante : le vote pondéré

Il paraît naturel qu'un voisin très proche compte plus qu'un voisin lointain. Dans le **vote pondéré**, chaque voisin reçoit un poids inversement proportionnel à sa distance :

$$\hat{y} = \operatorname{argmax}_{c \in C} \sum_{i \in N_k(x)} w_i \, \mathbf{1}(y_i = c), \qquad w_i = \frac{1}{d(x, x_i)}$$

EXEMPLE

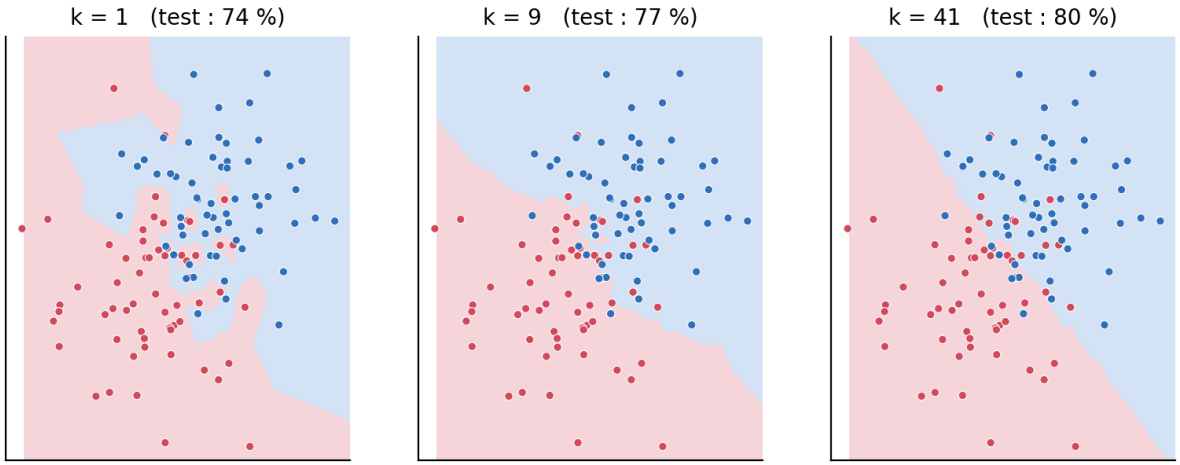
Pour k = 3 : poids de D = 1/1,41 = 0,71 (Refusé) ; poids de F = 1/2,00 = 0,50 et de E = 1/2,24 = 0,45 (Admis). Total Admis = 0,95 contre 0,71 pour Refusé : la prédiction reste Admis, avec une marge plus nuancée.

k plus proches voisins (k-NN)

Comment choisir k ?

Le paramètre k est le **réglage principal** de l'algorithme. Il pilote un compromis fondamental en apprentissage automatique.

k petit (k = 1) : la frontière est très découpée et épouse le moindre point, y compris le bruit. Le modèle **sur-apprend** (*overfitting*) : excellent sur les données d'apprentissage, médiocre sur de nouvelles données.



Zones de décision de k-NN pour k = 1, 9 et 41 (données simulées, deux classes qui se chevauchent). Les zones rouges et bleues indiquent la classe prédite.

k grand : la frontière devient lisse, mais le modèle ignore les détails locaux. Il **sous-apprend** (*underfitting*). Dans le cas extrême k = n, on prédit toujours la classe majoritaire du jeu de données.

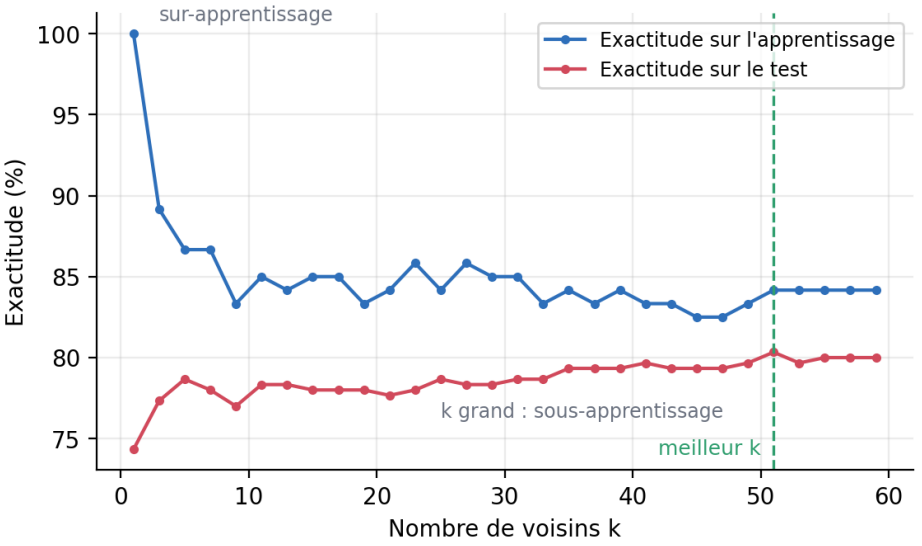
k plus proches voisins (k-NN)

Comment choisir k ?

Règles pratiques

En classification à **deux classes**, prendre k **impair** (3, 5, 7…) évite les égalités.

- Une heuristique de départ est k ≈ √n, où n est le nombre d'exemples d'apprentissage (pour n = 100 : k ≈ 10, on retient 9 ou 11).
- La méthode fiable est la **validation croisée** : on essaie plusieurs valeurs de k et l'on garde celle qui donne la meilleure exactitude moyenne sur des sous-ensembles de validation.



Rappel

La **validation croisée à K blocs** découpe les données en K parties de tailles égales. À tour de rôle, une partie sert de test et les K − 1 autres d'apprentissage. On calcule la moyenne des K scores obtenus.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
. . . . . .

k plus proches voisins (k-NN)

L'importance de la normalisation

Une distance additionne des écarts mesurés dans des **unités différentes**. Si une variable s'exprime en milliers (le salaire) et une autre en dizaines (l'âge), la première écrase la seconde dans le calcul : **l'algorithme ne « voit » plus que le salaire.**

EXEMPLE

Trois clients : **Client 1 (25 ans, 30 000 €), Client 2 (45 ans, 32 000 €), Client 3 (26 ans, 60 000 €).**

Distances brutes à partir du client 1 : vers le client 2 : $\sqrt{(20^2 + 2000^2)} \approx$ **2 000** ; vers le client 3 : $\sqrt{(1^2 + 30\,000^2)} \approx$ **30 000**. **Le client 2 paraît donc 15 fois plus proche, alors qu'il a 20 ans de plus !**

La solution est de **mettre toutes les variables à la même échelle**. Deux méthodes classiques :

• la **normalisation min-max**, qui ramène chaque variable dans l'intervalle [0, 1] :

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

• la **standardisation** (score z), qui centre sur 0 et réduit à un écart-type de 1 (μ est la moyenne et σ l'écart-type de la variable) :

$$z = \frac{x - \mu}{\sigma}$$

EXEMPLE

Avec des âges dans **[20 ; 60]** et des salaires dans **[20 000 ; 100 000],**

après min-max : **client 1 = (0,125 ; 0,125), client 2 = (0,625 ; 0,150), client 3 = (0,150 ; 0,500).**

Nouvelles distances depuis le client 1 : vers le client 2 : $\sqrt{(0{,}5^2 + 0{,}025^2)} \approx$ **0,50** ; vers le client 3 : $\sqrt{(0{,}025^2 + 0{,}375^2)} \approx$ **0,38**.

Le classement s'est **inversé** : le client 3 est désormais le plus proche.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
Naïve Bayes
.....

Naïve Bayes

Définition

Après une méthode géométrique (k-NN), voici une méthode **probabiliste**. Plutôt que de chercher des voisins, Naïve Bayes se demande : « Sachant les caractéristiques observées, **quelle est la probabilité** que l'exemple appartienne à chaque classe ? ». Il retient ensuite la classe la plus probable. Pour le comprendre, il faut d'abord se rafraîchir la mémoire sur quelques notions de probabilités.

Rappels de probabilités

Probabilité d'un événement

La **probabilité** P(A) d'un événement A est un nombre entre 0 (impossible) et 1 (certain) qui mesure sa chance de se produire. Si tous les cas sont équiprobables : P(A) = nombre de cas favorables / nombre de cas possibles.

EXEMPLE

Dans un jeu de 52 cartes, la probabilité de tirer un cœur est P(cœur) = 13 / 52 = **0,25**.



Chapitre 4: Classification

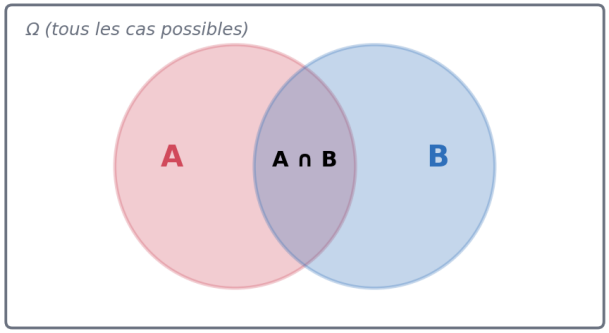
Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
Naïve Bayes
.....

Naïve Bayes

Probabilité conditionnelle

La **probabilité conditionnelle** de A sachant B, notée P(A | B), est la probabilité que A se réalise **lorsque l'on sait déjà que B s'est réalisé**. L'information B restreint l'univers des possibles.

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$



EXEMPLE

On tire une carte. Sachant que la carte est rouge (B), quelle est la probabilité que ce soit un cœur (A) ?

P(A ∩ B) = 13/52 (tous les cœurs sont rouges)
et
P(B) = 26/52,
donc P(A | B) = (13/52) / (26/52) = **0,5**.

Indépendance

Deux événements sont **indépendants** si savoir que l'un s'est produit ne change pas la probabilité de l'autre. Dans ce cas : $P(A \cap B) = P(A) \times P(B)$

EXEMPLE

Deux lancers successifs d'une pièce sont indépendants : P(pile puis pile) = 0,5 × 0,5 = **0,25**.

Chapitre 4: Classification
Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
Naïve Bayes
.....

Naïve Bayes

Théorème de Bayes

Le révérend Thomas Bayes (XVIII[e] siècle) a établi la formule qui permet de **renverser** une probabilité conditionnelle : à partir de P(X | C), « la probabilité d'observer X quand C est vrai », on obtient P(C | X), « la probabilité que C soit vrai quand on a observé X ».

$$P(C \mid X) = \frac{P(X \mid C) \times P(C)}{P(X)}$$

Terme	Nom	Signification
P(C)	Probabilité a priori (prior)	Ce que l'on croit de la classe C avant d'avoir observé quoi que ce soit
P(X | C)	Vraisemblance (likelihood)	À quel point l'observation X est typique de la classe C
P(X)	Évidence	Probabilité globale d'observer X, toutes classes confondues
P(C | X)	Probabilité a posteriori	Ce que l'on croit de C après avoir observé X : c'est la réponse cherchée

L'évidence se calcule en additionnant sur toutes les classes possibles (formule des probabilités totales) :

$$P(X) = \sum_{C} P(X \mid c)\, P(c)$$

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
Naïve Bayes
.....

Naïve Bayes

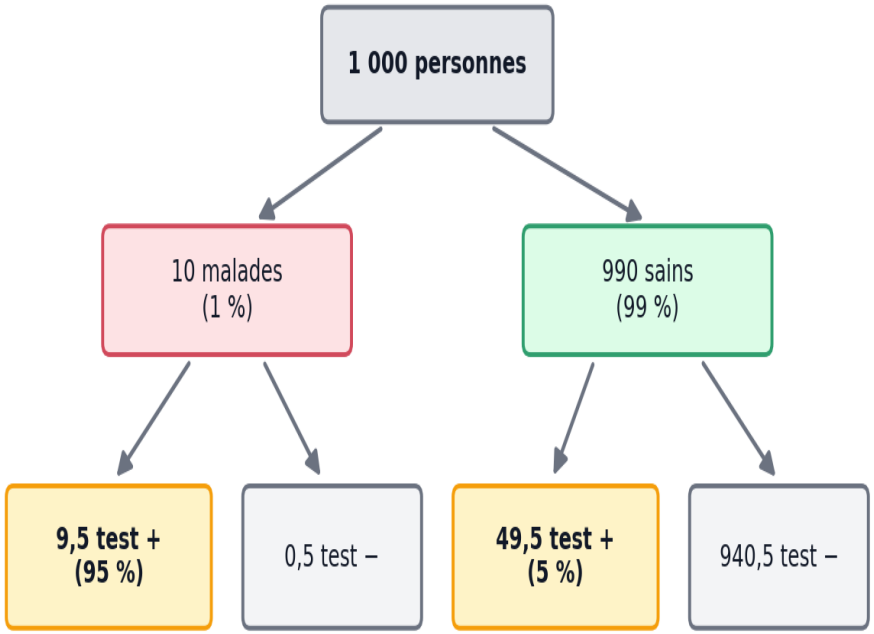
Théorème de Bayes

Un exemple célèbre : le test médical

EXEMPLE TEST MÉDICAL

Une maladie touche 1 % de la population. Un test la détecte chez 95 % des malades (sensibilité) et se trompe chez 5 % des personnes saines (faux positifs). Vous passez le test : il est positif. Quelle est la probabilité que vous soyez réellement malade ?

Presque tout le monde répond « environ 95 % ». C'est faux ! Imaginons 1 000 personnes pour y voir clair (on parle de *fréquences naturelles*).



Parmi les 59 tests positifs (9,5 + 49,5), seuls 9,5 concernent de vrais malades : 9,5 / 59 ≈ 16 %

Naïve Bayes

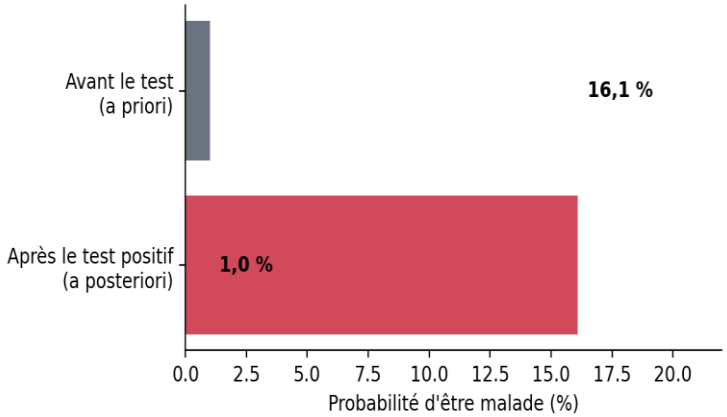
Théorème de Bayes

Un exemple célèbre : le test médical

Parmi les 1 000 personnes, 10 sont malades et 9,5 d'entre elles ont un test positif. Parmi les 990 personnes saines, 5 % soit 49,5 ont aussi un test positif. Au total 59 tests sont positifs, dont seulement 9,5 sont de vrais malades. Avec la formule de Bayes, on retrouve le même résultat :

$$P(C \mid X) = \frac{P(X \mid C) \times P(C)}{P(X)} \qquad P(X) = \sum_c P(X \mid c)\, P(c)$$

$$P(M \mid +) = \frac{0{,}95 \times 0{,}01}{0{,}95 \times 0{,}01 + 0{,}05 \times 0{,}99} = \frac{0{,}0095}{0{,}0590} \approx 0{,}16$$



Le test positif fait passer la probabilité d'être malade de 1 % à environ 16 %.

À RETENIR

La probabilité a posteriori dépend **à la fois** de la vraisemblance (le test est fiable) **et** de la probabilité a priori (la maladie est rare). Ignorer l'a priori est une erreur très fréquente, appelée *oubli du taux de base*.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
Naïve Bayes
……

Naïve Bayes

Théorème de Bayes au classifieur

Appliquons Bayes à la classification. L'observation X est le vecteur des caractéristiques (x_1, …, x_p) d'un exemple, et C est sa classe. Pour chaque classe c, on veut calculer P(c | x_1, …, x_p), puis choisir la classe qui a la plus forte probabilité a posteriori : c'est la règle du **maximum a posteriori** (MAP).

INTUITION

Le dénominateur P(X) est le même pour toutes les classes. Pour comparer les classes entre elles, on peut donc l'ignorer : il suffit de comparer les numérateurs P(X | c) × P(c).

Le problème de la vraisemblance conjointe

Le terme P(x_1, …, x_p | c) est difficile à estimer. Avec p variables binaires, il y a 2^p combinaisons possibles : pour p = 30, plus d'un milliard ! On n'aura jamais assez d'exemples pour observer chaque combinaison.

Naïve Bayes

L'hypothèse « naïve »

Naïve Bayes lève cette difficulté par une hypothèse simplificatrice : **sachant la classe, les caractéristiques sont indépendantes les unes des autres**. La probabilité conjointe se factorise alors en un produit de probabilités simples

$$P(x_1, x_2, \ldots, x_p \mid c) = \prod_{j=1}^{p} P(x_j \mid c)$$

D'où la règle de décision de Naïve Bayes : $\hat{y} = \text{argmax}_{c \in C}\, P(c) \prod_{j=1}^{p} P(x_j \mid c)$

DÉFINITION NAÏVE BAYES

Un classifieur Naïve Bayes choisit la classe qui maximise le produit de la probabilité a priori P(c) et des vraisemblances $P(x_j \mid c)$ de chaque caractéristique, calculées séparément.

ATTENTION

L'hypothèse d'indépendance est « naïve » car elle est presque toujours fausse dans la réalité (la taille et le poids d'une personne sont liés, par exemple). Pourtant, le classifieur fonctionne souvent très bien : il lui suffit que la classe la plus probable soit la bonne, pas que les probabilités soient exactes.

Naïve Bayes

L'astuce du logarithme

Multiplier de nombreuses probabilités inférieures à 1 produit un résultat minuscule que l'ordinateur finit par arrondir à 0 (*underflow*). On passe donc aux **logarithmes** : comme ln est strictement croissante, le maximum reste le même, et les produits deviennent des sommes.

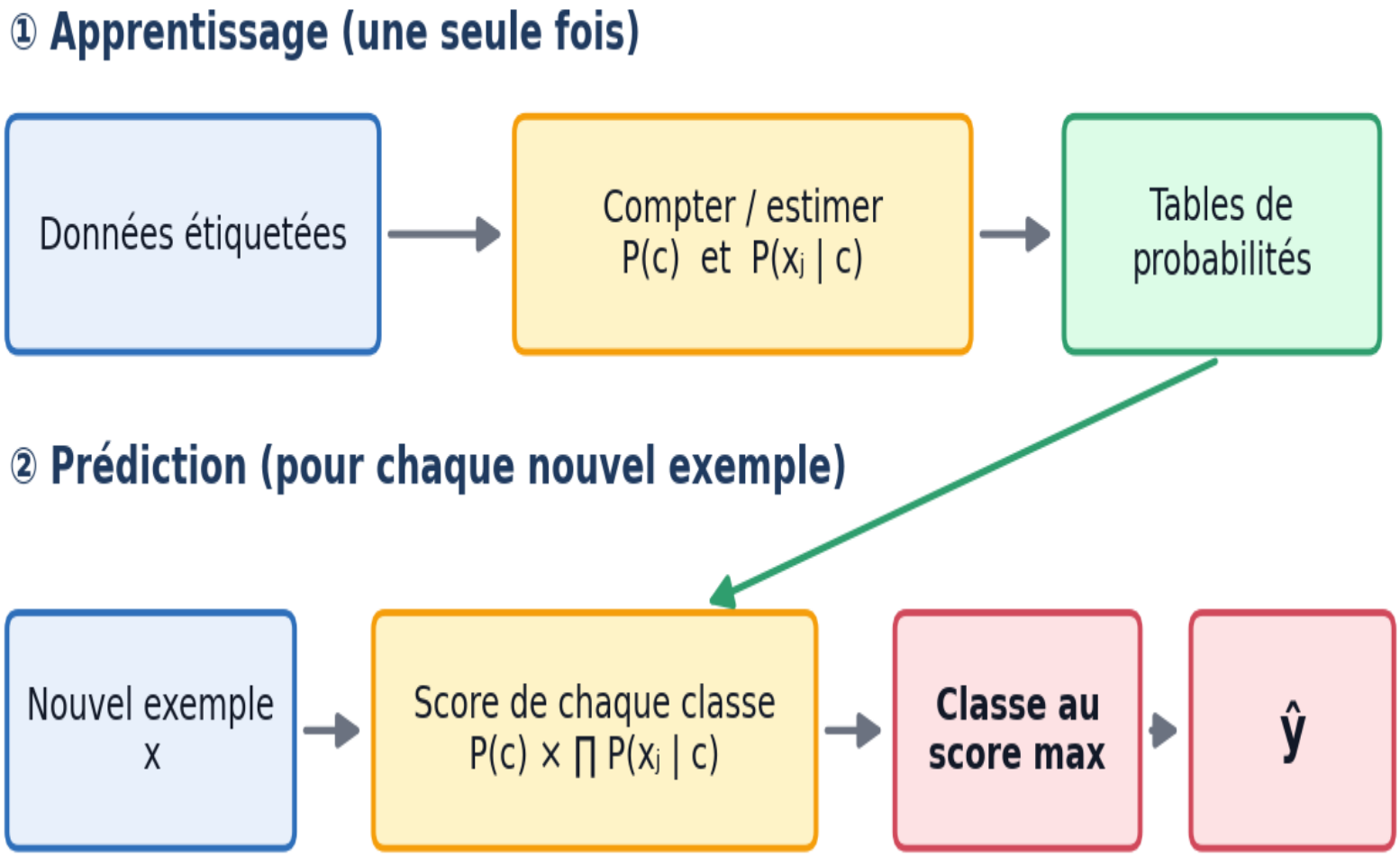
$$\hat{y} = \text{argmax}_{c \in C} \left[\ln P(c) + \sum_{j=1}^{p} \ln P(x_j \mid c) \right]$$

Apprentissage et prédiction

« Entraîner » un Naïve Bayes revient à **compter** dans le jeu d'apprentissage. On estime :
les probabilités a priori : la proportion de chaque classe ;
les vraisemblances : la proportion de chaque valeur de chaque variable à l'intérieur de chaque classe.

$$P(c) = \frac{n_c}{n} \qquad P(x_j = v \mid c) = \frac{n_{v,c}}{n_c}$$

où n_c est le nombre d'exemples de la classe c, n le nombre total d'exemples et n_v, c le nombre d'exemples de la classe c pour lesquels la variable x_j vaut v.


① Apprentissage (une seule fois)
Données étiquetées
Compter / estimer
P(c) et P(xⱼ | c)
Tables de probabilités
② Prédiction (pour chaque nouvel exemple)
Nouvel exemple
x
Score de chaque classe
P(c) × ∏ P(xⱼ | c)
Classe au score max
ŷ

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
Naïve Bayes
……

Naïve Bayes

Exemple complet : jouer au tennis ?

Un club décide chaque matin si les courts sont ouverts selon la météo. Voici l'historique de 14 journées (jeu de données classique d'initiation).

Question.

Demain :

→ ciel **ensoleillé**,
→ température **fraîche**,
→ humidité **élevée**,
→ vent **fort**.

Joue-t-on ?

N°	Ciel	Température	Humidité	Vent	Jouer ?
1	**Ensoleillé**	**Chaud**	**Élevée**	Faible	Non
2	**Ensoleillé**	**Chaud**	**Élevée**	**Forte**	Non
3	Couvert	**Chaud**	**Élevée**	Faible	Oui
4	Pluvieux	Doux	**Élevée**	Faible	Oui
5	Pluvieux	Frais	Normale	Faible	Oui
6	Pluvieux	Frais	Normale	**Forte**	Non
7	Couvert	Frais	Normale	**Forte**	Oui
8	**Ensoleillé**	Doux	**Élevée**	Faible	Non
9	**Ensoleillé**	Frais	Normale	Faible	Oui
10	Pluvieux	Doux	Normale	Faible	Oui
11	**Ensoleillé**	Doux	Normale	**Forte**	Oui
12	Couvert	Doux	**Élevée**	**Forte**	Oui
13	Couvert	**Chaud**	Normale	Faible	Oui
14	Pluvieux	Doux	**Élevée**	**Forte**	Non

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
Naïve Bayes
.....

Naïve Bayes

Exemple complet : jouer au tennis ?

Étape 1 Les probabilités a priori

Sur 14 jours, on a joué 9 fois (Oui) et pas joué 5 fois (Non) : P(Oui) = 9/14 ≈ 0,643 et P(Non) = 5/14 ≈ 0,357.

Étape 2 Les vraisemblances (comptages par classe)

Table Vraisemblances estimées par comptage

Variable (X)= valeur	Oui (sur 9)	Non (sur 5)	P(· | Oui)	P(· | Non)
Ciel = Ensoleillé	2	3	2/9	3/5
Température = Frais	3	1	3/9	1/5
Humidité = Élevée	3	4	3/9	4/5
Vent = Fort	3	3	3/9	3/5

Étape 3 Le score de chaque classe

On multiplie l'a priori par les quatre vraisemblances (S désigne le score non normalisé) :

$$S(C) = P(C) \times P(X \mid C)$$

$$P(x_1, x_2, \ldots, x_p \mid c) = \prod_{j=1}^{p} P(x_j \mid c)$$

$$S(\text{Oui}) = \frac{9}{14} \times \frac{2}{9} \times \frac{3}{9} \times \frac{3}{9} \times \frac{3}{9} \approx 0{,}0053$$

$$S(\text{Non}) = \frac{5}{14} \times \frac{3}{5} \times \frac{1}{5} \times \frac{4}{5} \times \frac{3}{5} \approx 0{,}0206$$

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
Naïve Bayes
.....

Naïve Bayes

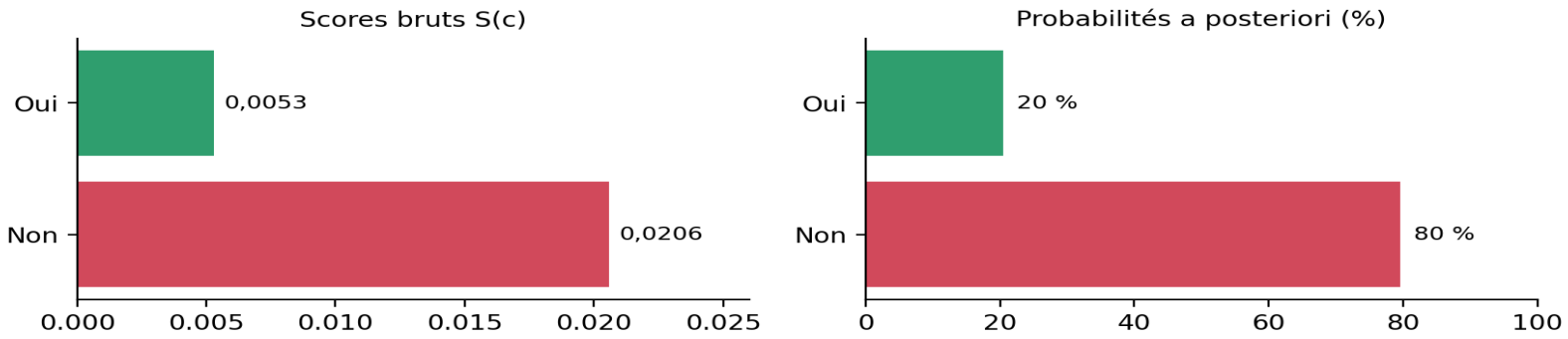
Exemple complet : jouer au tennis ?

Étape 4 Normalisation et décision

Les deux scores ne sont pas des probabilités (leur somme n'est pas 1). Pour obtenir les probabilités a posteriori, on divise chaque score par leur somme, c'est-à-dire par l'évidence P(X) :

$$P(\text{Oui} \mid x) = \frac{0{,}0053}{0{,}0053 + 0{,}0206} \approx 0{,}20 \qquad P(\text{Non} \mid x) \approx 0{,}80$$

Exemple : (Ensoleillé, Frais, Humidité élevée, Vent fort)



Prédiction : Non, avec environ 80 % de confiance. **Le ciel ensoleillé, l'humidité élevée et le vent fort sont, dans l'historique, plus souvent associés à des journées sans tennis.**

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
Naïve Bayes
.....

Naïve Bayes

Problème de la probabilité nulle et le lissage de Laplace

Dans le table du dataset, il n'existe aucune journée « Couvert » avec la réponse « Non » : P(Couvert | Non) = 0/5 = **0**. Si l'on reçoit une nouvelle journée couverte, le score de « Non » sera multiplié par 0, donc **égal à 0, quelles que soient les autres variables**. Une seule valeur jamais vue avec une classe suffit à l'éliminer définitivement !
Le **lissage de Laplace** (ou lissage additif) corrige cela en ajoutant un petit nombre α (en général 1) à chaque comptage, comme si l'on avait vu chaque valeur au moins α fois :

$$P(x_j = v \mid c) = \frac{n_{v,c} + \alpha}{n_c + \alpha\, m_j}$$

où m_j est le nombre de valeurs possibles de la variable x_j. On ajoute $\alpha \cdot m_j$ au dénominateur pour que les probabilités continuent à totaliser 1.

EXEMPLE LISSAGE

Pour la variable Ciel **(m = 3 valeurs)** avec α = 1 : **P(Couvert | Non) = (0 + 1) / (5 + 3) = 0,125** au lieu de 0.
Pour **P(Couvert | Oui) = (4 + 1) / (9 + 3) = 5/12 ≈ 0,417** au lieu de 4/9 ≈ 0,444. Les probabilités sont légèrement ajustées, mais plus aucune n'est nulle.

Naïve Bayes

Variables continues : Naïve Bayes gaussien

Que faire si une variable est un **nombre réel** (poids, température mesurée, revenu) ? On ne peut plus compter les valeurs exactes. L'approche classique suppose que, dans chaque classe, la variable suit une **loi normale** (gaussienne) de moyenne μ et de variance σ², estimées sur les données :

$$\mu_{jc} = \frac{1}{n_c} \sum_{i\,:\,y_i = c} x_{ij} \qquad \sigma^2_{jc} = \frac{1}{n_c} \sum_{i\,:\,y_i = c} (x_{ij} - \mu_{jc})^2$$

La vraisemblance est alors la valeur de la densité de la loi normale au point observé :

$$P(x_j \mid c) = \frac{1}{\sqrt{2\pi\sigma^2_{jc}}} \exp\left(-\frac{(x_j - \mu_{jc})^2}{2\sigma^2_{jc}}\right)$$

EXEMPLE

On veut distinguer des mandarines (poids moyen μ = 80 g, écart-type σ = 10 g) et des oranges (μ = 150 g, σ = 20 g). Les deux classes sont également fréquentes (P = 0,5).

Un fruit pèse 100 g : de quelle classe est-il ?

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
Naïve Bayes
.....

Naïve Bayes

Variables continues : Naïve Bayes gaussien

EXEMPLE

On veut distinguer des mandarines (poids moyen μ = 80 g, écart-type σ = 10 g) et des oranges (μ = 150 g, σ = 20 g). Les deux classes sont également fréquentes (P = 0,5).

Un fruit pèse 100 g : de quelle classe est-il ?

$$P(x_j \mid c) = \frac{1}{\sqrt{2\pi\sigma_{jc}^2}} \exp\left(-\frac{(x_j - \mu_{jc})^2}{2\sigma_{jc}^2}\right)$$

$$P(x = 100 \mid \text{Mandarine}) = \frac{1}{10\sqrt{2\pi}} \exp\left(-\frac{(100-80)^2}{2\times 10^2}\right) \approx 0{,}0054$$

$$P(x = 100 \mid \text{Orange}) = \frac{1}{20\sqrt{2\pi}} \exp\left(-\frac{(100-150)^2}{2\times 20^2}\right) \approx 0{,}0009$$

Comme les a priori sont égaux, la comparaison porte sur les vraisemblances

$$P(\textbf{Mandarine} \mid 100) = \frac{P(100 \mid \textbf{Mandarine})P(\textbf{Mandarine})}{P(100 \mid \textbf{Mandarine})P(\textbf{Mandarine}) + P(100 \mid \boldsymbol{Orange})P(\boldsymbol{Orange})}$$

En normalisant : **P(Mandarine | 100 g) = 0,0054x0,5 / (0,0054 x0,5+ 0,0009x0,5) ≈ 0,86.**
Le fruit est donc classé « mandarine », bien qu'il soit relativement lourd pour cette classe, parce que 100 g reste beaucoup plus plausible pour une mandarine que pour une orange.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
Naïve Bayes
.....

Naïve Bayes

Variables continues : Naïve Bayes gaussien

EXEMPLE

On veut distinguer des mandarines (poids moyen μ = 80 g, écart-type σ = 10 g) et des oranges (μ = 150 g, σ = 20 g). Les deux classes sont également fréquentes (P = 0,5).

Un fruit pèse 100 g : de quelle classe est-il ?

$$P(x_j \mid c) = \frac{1}{\sqrt{2\pi\sigma_{jc}^2}} \exp\left(-\frac{(x_j - \mu_{jc})^2}{2\sigma_{jc}^2}\right)$$

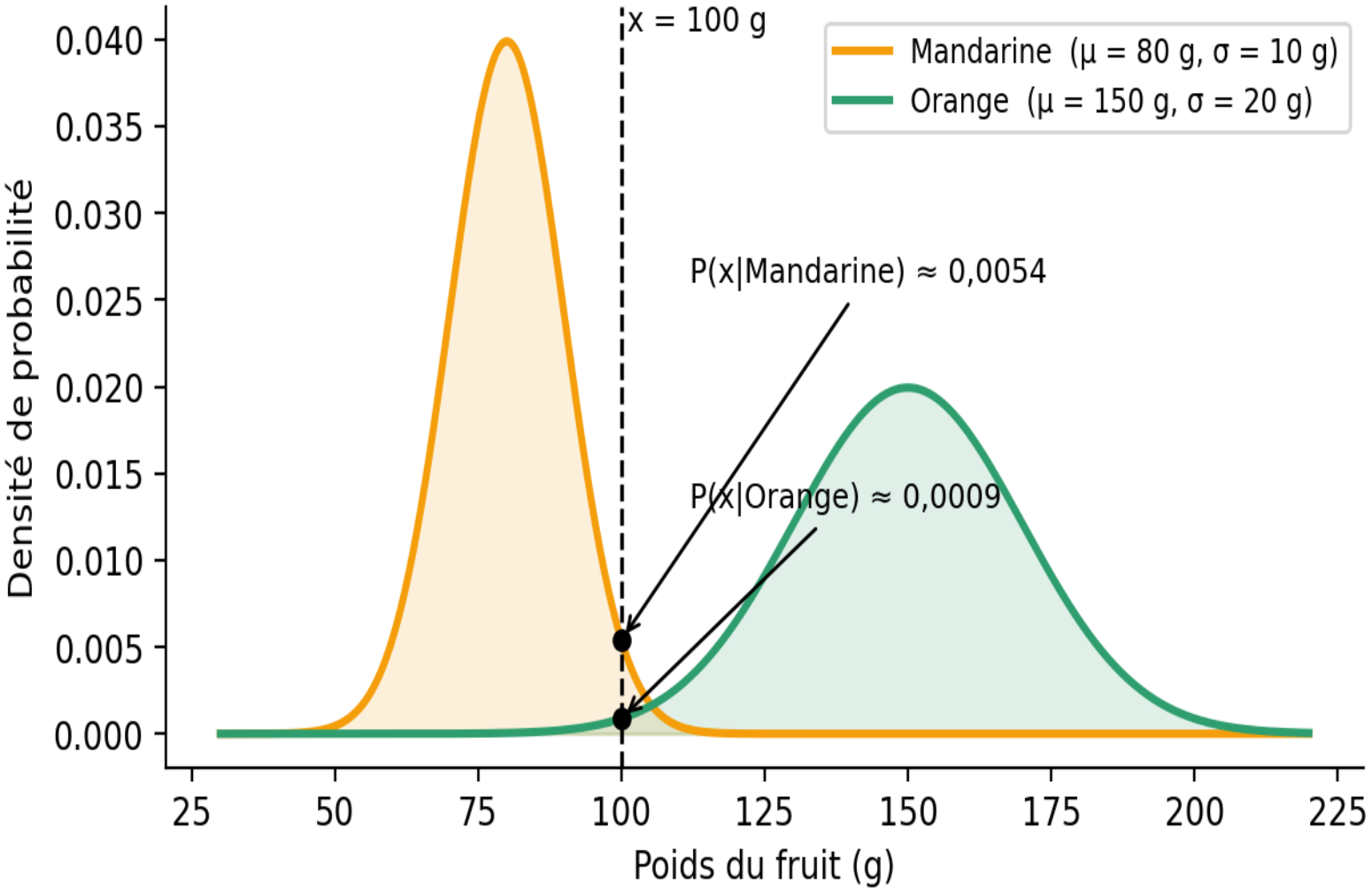
$$P(x = 100 \mid \text{Mandarine}) = \frac{1}{10\sqrt{2\pi}} \exp\left(-\frac{(100-80)^2}{2\times 10^2}\right) \approx 0{,}0054$$

$$P(x = 100 \mid \text{Orange}) = \frac{1}{20\sqrt{2\pi}} \exp\left(-\frac{(100-150)^2}{2\times 20^2}\right) \approx 0{,}0009$$

Comme les a priori sont égaux, la comparaison porte sur les vraisemblances

$$P(\textbf{Mandarine} \mid 100) = \frac{P(100 \mid \textbf{Mandarine})P(\textbf{Mandarine})}{P(100 \mid \textbf{Mandarine})P(\textbf{Mandarine}) + P(100 \mid \boldsymbol{Orange})P(\boldsymbol{Orange})}$$

En normalisant : **P(Mandarine | 100 g) = 0,0054x0,5 / (0,0054 x0,5+ 0,0009x0,5) ≈ 0,86.**
Le fruit est donc classé « mandarine », bien qu'il soit relativement lourd pour cette classe, parce que 100 g reste beaucoup plus plausible pour une mandarine que pour une orange.



Avec plusieurs variables continues, on multiplie simplement les densités de chacune, comme dans le cas catégoriel.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
Naïve Bayes
……

Naïve Bayes

Forces et faiblesses

Avantages	Inconvénients
Apprentissage et prédiction extrêmement rapides (simples comptages)	L'hypothèse d'indépendance est rarement vraie ; des variables très corrélées sont « comptées plusieurs fois »
Fonctionne avec peu de données d'apprentissage	Les probabilités fournies sont souvent mal calibrées (trop proches de 0 ou de 1)
Supporte un très grand nombre de variables (textes)	Il faut un lissage pour les valeurs jamais vues
Facile à interpréter : on voit la contribution de chaque variable	Pour les variables continues, l'hypothèse gaussienne peut être inadaptée
Gère naturellement plusieurs classes	Frontières de décision limitées (pas d'interactions entre variables)

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
Naïve Bayes
.....

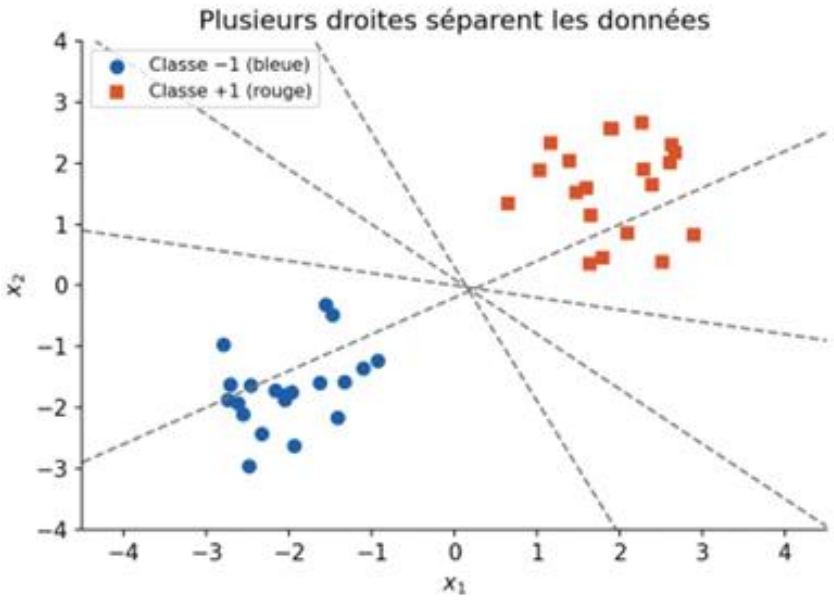
SVM

L'idée intuitive : la marge maximale

Une infinité de frontières possibles

Considérons deux groupes de points linéairement séparables, c'est-à-dire que l'on peut séparer par une droite (en dimension 2) ou, plus généralement, par un hyperplan (en dimension p).

Combien de droites séparent correctement ces points ? Une infinité



Toutes ces droites commettent zéro erreur sur les données d'entraînement. Pourtant, elles ne se valent pas : une droite qui frôle un point est fragile. Si un nouvel exemple est légèrement différent (du bruit de mesure, par exemple), il risque de tomber du mauvais côté

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
Naïve Bayes
……

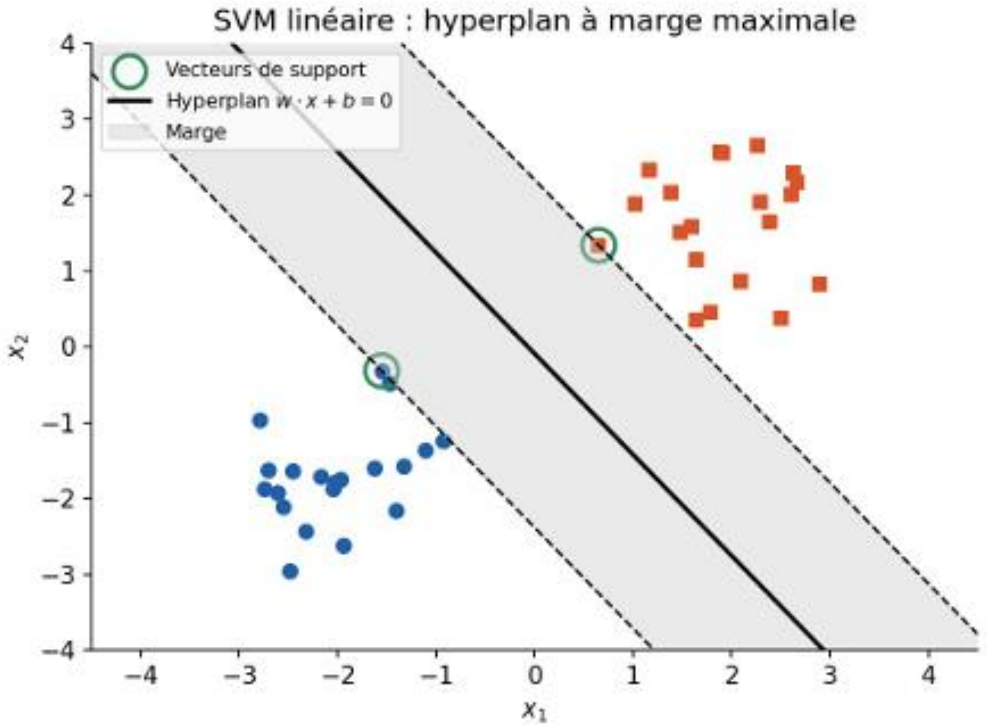
SVM

La marge : laisser le maximum de place

L'idée géniale des SVM est de choisir la frontière qui est la plus éloignée possible des deux classes. Imaginez que vous tracez une route entre deux villages : vous la voulez aussi large que possible, sans toucher aucune maison. La largeur de cette route s'appelle la marge

Définition

Marge et vecteurs de support La marge est la distance entre l'hyperplan séparateur et les exemples d'entraînement les plus proches (ou, selon la convention, la largeur totale de la « route » de part et d'autre de l'hyperplan). Les vecteurs de support sont les exemples situés exactement sur le bord de la marge. Ce sont les seuls points qui « soutiennent » (déterminent) l'hyperplan : si on supprime un autre point, la solution ne change pas



Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
Naïve Bayes
.....

SVM

SVM linéaire à marge dure

Nous traduisons maintenant l'idée de marge en mathématiques. On parle de marge dure (hard margin) quand on exige que tous les exemples soient correctement classés et situés en dehors de la marge. Cela n'est possible que si les données sont linéairement séparables

L'hyperplan et la fonction de décision

Un hyperplan est l'ensemble des points x qui vérifient l'équation :

$$\mathbf{w} \cdot \mathbf{x} + b = 0$$

En dimension 2, c'est l'équation d'une droite : $w_1 x_1 + w_2 x_2 + b = 0$. Le vecteur w est perpendiculaire à l'hyperplan et b le décale par rapport à l'origine. Un point est classé selon le signe de la quantité w·x + b

$$f(\mathbf{x}) = \text{sign}\,(\mathbf{w} \cdot \mathbf{x} + b)$$

- si $\mathbf{w}\cdot\mathbf{x} + b > 0$, le point est du côté « positif » et on prédit la classe **+1** ;
- si $\mathbf{w}\cdot\mathbf{x} + b < 0$, le point est du côté « négatif » et on prédit la classe **−1** ;
- si $\mathbf{w}\cdot\mathbf{x} + b = 0$, le point est exactement sur la frontière.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
Naïve Bayes
.....

SVM

Distance d'un point à l'hyperplan

La distance entre un point x et l'hyperplan est donnée par la formule suivante. Plus cette valeur est grande, plus on est « sûr » de la classe du point

$$d(\mathbf{x}) = \frac{|\mathbf{w} \cdot \mathbf{x} + b|}{\|\mathbf{w}\|}$$

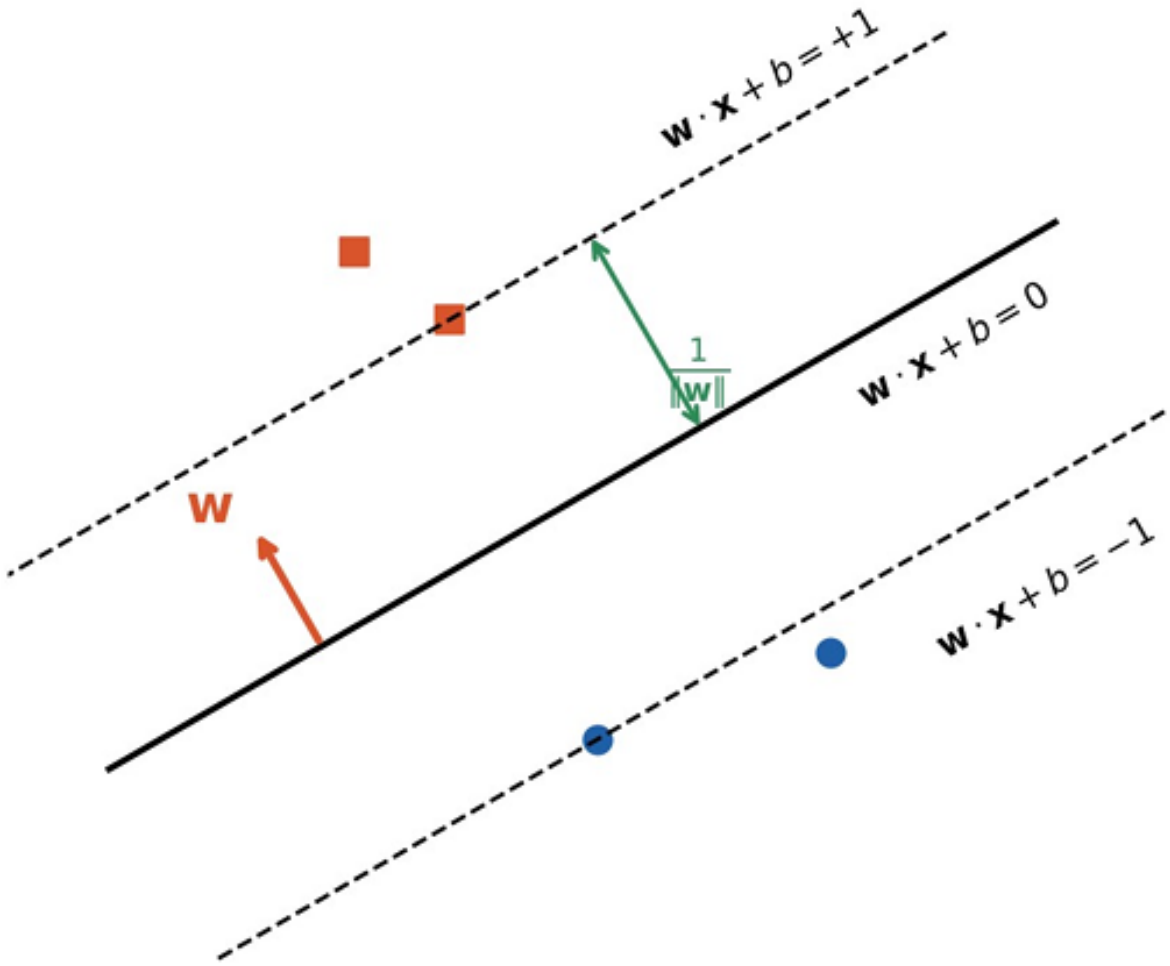
Imposer la marge : les contraintes

Les équations w·x + b = 0, 2w·x + 2b = 0 ou 10w·x + 10b = 0 décrivent le même hyperplan : on peut multiplier (w, b) par un nombre positif sans rien changer. On utilise cette liberté pour fixer l'échelle de sorte que les points les plus proches (les vecteurs de support) vérifient exactement w·x + b = +1 ou −1. La condition de bonne classification avec marge s'écrit alors, pour tous les exemples :

$$y_i (\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 \quad \text{pour } i = 1, \ldots, n$$

En effet : pour un exemple de classe +1, on veut w·xi + b ≥ 1 ; pour un exemple de classe −1, on veut w·xi + b ≤ −1. Multiplier par yi (qui vaut +1 ou −1) regroupe les deux cas en une seule inégalité.

Géométrie du SVM : le vecteur w est perpendiculaire à l'hyperplan ; la distance de l'hyperplan à chaque bord de la marge est 1/‖w‖



Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
Naïve Bayes
……

SVM

La largeur de la marge

Sur la figure précedente, la distance de l'hyperplan à l'un des bords vaut 1/‖w‖ ; la largeur totale de la route est donc :

$$\text{marge} = \frac{2}{\|\mathbf{w}\|}$$

Idée clé

Maximiser la marge 2/‖w‖ revient à minimiser ‖w‖ (ou, ce qui est équivalent et plus pratique pour le calcul, à minimiser ½‖w‖²). Un vecteur w « petit » donne une marge « grande »

Problème d'optimisation

Le SVM à marge dure est le problème d'optimisation suivant :
trouver (w, b) qui minimisent la quantité $\rightarrow \min_{\mathbf{w},\, b} \frac{1}{2}\|\mathbf{w}\|^2$

sous les contraintes valables pour tous les exemples. C'est un problème d'optimisation convexe (une fonction en forme de bol, avec des contraintes linéaires) :
il **possède une solution unique** **et on sait la calculer efficacement**. C'est l'un des grands atouts des SVM par rapport, par exemple, aux réseaux de neurones, dont l'entraînement peut rester coincé dans des minima locaux.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
Naïve Bayes
.....

SVM

Exemple entièrement calculé à la main

Prenons quatre points dans le plan, deux par classe :

Point	Coordonnées	Classe y
A	(0 ; 0)	−1
B	(1 ; 1)	−1
C	(3 ; 3)	+1
D	(4 ; 4)	+1

Intuitivement, **les points proches de la frontière sont B et C** ; A et D sont « loin derrière ». On s'attend donc à ce que B et C soient les vecteurs de support. Les quatre points étant alignés sur la diagonale, w est parallèle à (1 ; 1) : posons w = (a ; a). Les vecteurs de support vérifient w·x + b = ±1

$$\mathbf{w} \cdot C + b = 1 \Rightarrow 6a + b = 1 \qquad \mathbf{w} \cdot B + b = -1 \Rightarrow 2a + b = -1$$

En soustrayant les deux équations : 4a = 2, donc a = 0,5, puis b = −1 − 2(0,5) = −2. On obtient :

$$\mathbf{w} = (0{,}5\ ;\ 0{,}5), \qquad b = -2$$

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
Naïve Bayes
.....

SVM

Exemple entièrement calculé à la main

La fonction de décision est donc :

$$f_0(\mathbf{x}) = 0{,}5\,x_1 + 0{,}5\,x_2 - 2$$

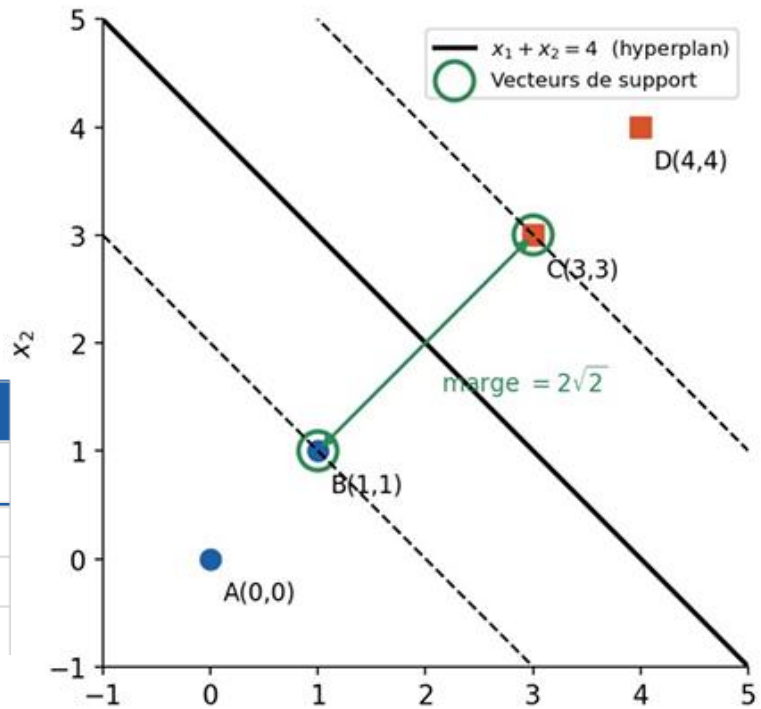
L'hyperplan a pour équation x1 + x2 = 4. Calculons la marge :

$$\text{marge} = \frac{2}{\|\mathbf{w}\|} = \frac{2}{\sqrt{0{,}5^2 + 0{,}5^2}} = \frac{2}{0{,}707} \approx 2{,}83 = 2\sqrt{2}$$

solution:

- B(1,1) et C(3,3) sont les vecteurs de support ;
- la marge vaut 2√2 ≈ 2,83.

Point	$f_0(x) = 0{,}5x_1 + 0{,}5x_2 - 2$	$y \cdot f_0(x)$	Rôle
A (0 ; 0)	−2	+2 (≥ 1)	bien classé, hors marge
B (1 ; 1)	−1	+1 (= 1)	**vecteur de support**
C (3 ; 3)	+1	+1 (= 1)	**vecteur de support**
D (4 ; 4)	+2	+2 (≥ 1)	bien classé, hors marge



Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
Naïve Bayes
.....

SVM

SVM à marge souple et paramètre C

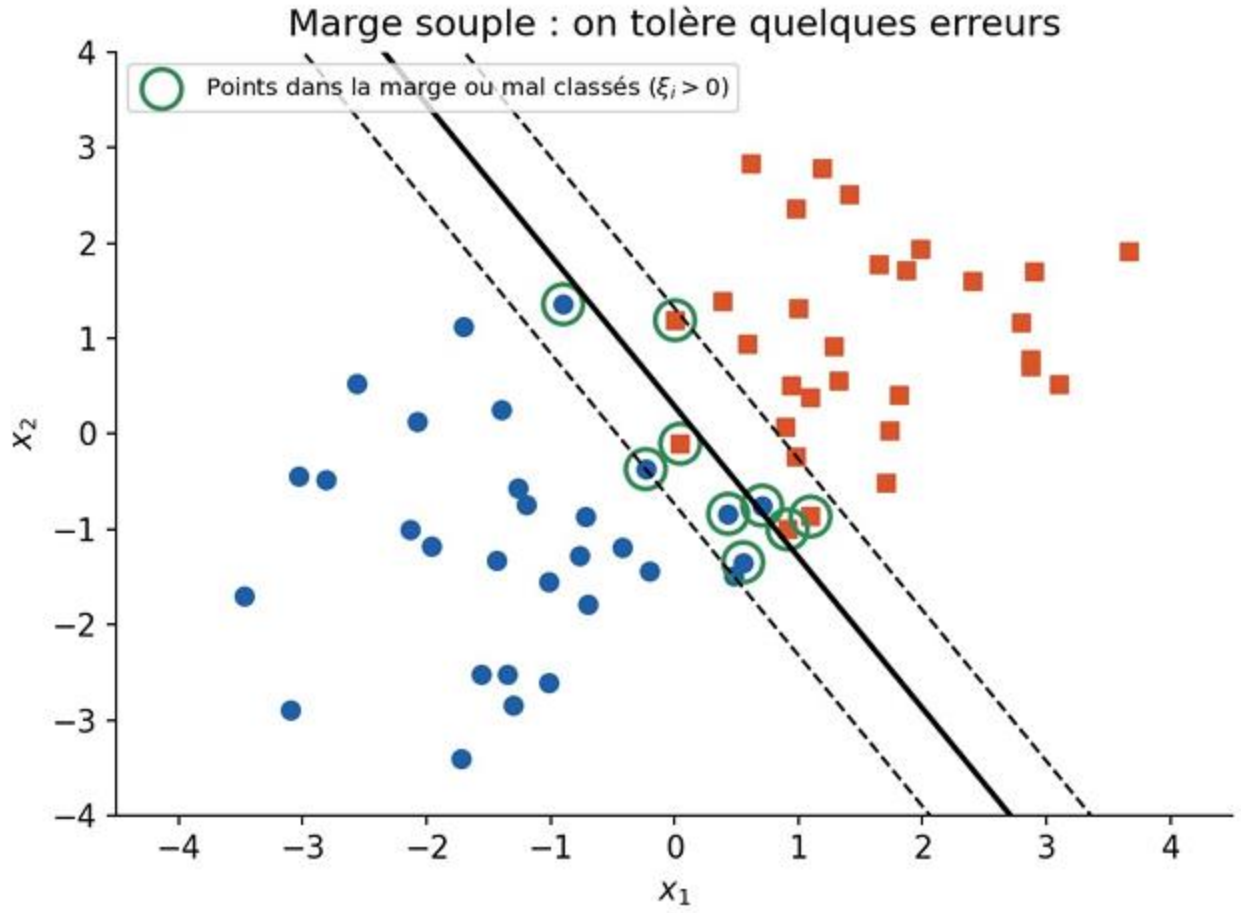
Dans la vraie vie, les données ne sont presque jamais parfaitement séparables : il y a du bruit, des erreurs d'étiquetage, des points atypiques (outliers). Exiger zéro erreur serait impossible, ou conduirait à une frontière absurde, tordue pour satisfaire un seul point aberrant. La solution est d'assouplir les contraintes : c'est la marge souple (soft margin).

Les variables ressort

On autorise chaque exemple i à violer la marge d'une quantité ξi ≥ 0, appelée variable ressort (slack variable)

$$y_i\,(\mathbf{w}\cdot\mathbf{x}_i + b) \geq 1 - \xi_i \quad \text{et} \quad \xi_i \geq 0$$

- $\xi_i = 0$: le point est bien classé et en dehors de la marge ;
- $0 < \xi_i < 1$: le point est bien classé mais **à l'intérieur de la marge** ;
- $\xi_i > 1$: le point est du **mauvais côté** de l'hyperplan (erreur de classification).



Marge souple : les points cerclés ont une variable ressort ξ > 0 (dans la marge ou mal classés).

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
Naïve Bayes
……

SVM

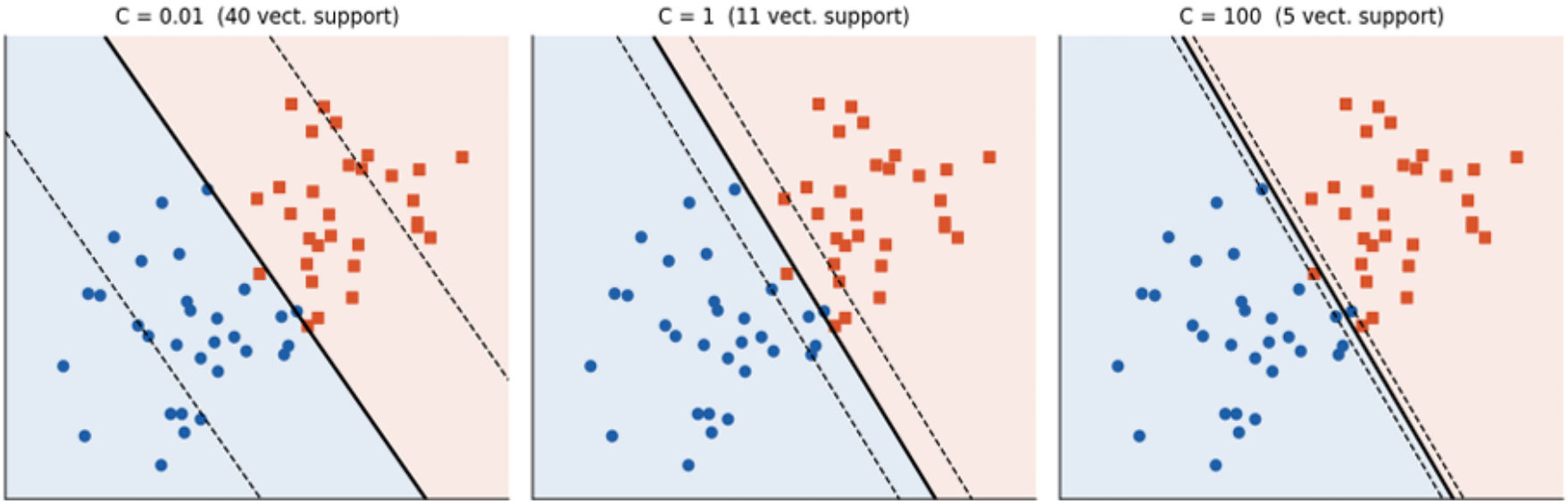
SVM à marge souple et paramètre C

Bien sûr, on ne veut pas autoriser n'importe quelles violations : on les pénalise dans la fonction à minimiser. Le problème devient :

$$\min_{\mathbf{w},\, b,\, \xi} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i$$

Cette expression combine deux objectifs contradictoires. Le premier terme, ½‖w‖², pousse vers une grande marge. Le second, C Σ ξi, pousse vers peu d'erreurs. Le paramètre C > 0 règle le compromis

	C petit (ex. 0,01)	C grand (ex. 100)
Priorité	grande marge, tolère beaucoup d'erreurs	peu d'erreurs, marge étroite
Frontière	simple, lisse	complexe, colle aux données
Vecteurs de support	nombreux	moins nombreux
Risque	sous-apprentissage (modèle trop simple)	sur-apprentissage (apprend le bruit)



Analogie

Le professeur et les élèves, Imaginez que C est la sévérité d'un correcteur. Avec un C énorme, il ne tolère aucune faute et réécrit tout le cours pour qu'aucun élève ne se trompe, même sur une question mal posée. Avec un C minuscule, il laisse passer beaucoup d'erreurs pour garder un cours simple. Le bon C est entre les deux.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
Naïve Bayes
……

SVM

SVM à marge souple et paramètre C

Bien sûr, on ne veut pas autoriser n'importe quelles violations : on les pénalise dans la fonction à minimiser. Le problème devient :

$$\min_{\mathbf{w},\, b,\, \xi} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i$$

Cette expression combine deux objectifs contradictoires. Le premier terme, ½‖w‖², pousse vers une grande marge. Le second, C Σ ξi, pousse vers peu d'erreurs. Le paramètre C > 0 règle le compromis

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
Naïve Bayes
……

SVM

Algorithme d'apprentissage : étapes

Deux éléments : maximiser la marge et pénaliser les violations.

$$\min \ \tfrac{1}{2} ||w||^2 + C \sum_i \xi_i$$

$$\text{sous contraintes : } \quad y_i (w^T x_i + b) \geq 1 - \xi_i$$

$$\xi_i \geq 0 \quad \text{pour tout } i$$

Le modèle recherche un compromis : ne pas trop réduire la marge pour corriger chaque point difficile, mais ne pas accepter trop de violations.

Interprétation de l'objectif

$\frac{1}{2}||w||^2$: favorise une marge large.

$C\Sigma\xi_i$: pénalise les violations.

La somme $\Sigma\xi_i$ additionne les violations sur toutes les observations d'apprentissage.

Les symboles de la formulation doivent être compris avant l'algorithme.

x_i	Vecteur des caractéristiques de l'observation i (ses variables d'entrée).	y_i	Étiquette de classe : +1 ou −1 en classification binaire.
w	Vecteur des poids : il détermine l'orientation de l'hyperplan.	b	Biais (interception) : il déplace la frontière.
$\xi_i \geq 0$	Variable d'écart (slack) propre à l'observation i ; elle quantifie la violation.	$C > 0$	Paramètre de régularisation ; il fixe le coût attribué aux violations.

i désigne une observation parmi les n observations d'apprentissage.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
Naïve Bayes
……

SVM

Algorithme d'apprentissage : étapes

Que représente exactement ξ_i ?

On calcule d'abord la marge fonctionnelle $z_i = y_i f(x_i)$.

$$\xi_i = \max(0, 1 - y_i(w^T x_i + b))$$

$\xi_i = 0$
$z_i \geq 1$
Point bien classé et situé sur ou au-delà de la marge.

$0 < \xi_i < 1$
$0 < z_i < 1$
Point bien classé, mais à l'intérieur de la marge.

$\xi_i > 1$
$z_i < 0$
Point du mauvais côté de la frontière : erreur de classification.

Attention : $\xi_i = 1$ correspond à $z_i = 0$, c'est-à-dire un point exactement sur la frontière de décision. ξ_i peut être supérieur à 1 si le point est mal classé.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
Naïve Bayes
……

SVM

Algorithme d'apprentissage : étapes

1	**Préparer les données**	Chaque observation est représentée par x_i et son étiquette $y_i \in \{-1,+1\}$.
2	**Choisir C et le noyau**	Pour le SVM linéaire, on utilise directement $w^T x+b$. Pour un problème non linéaire, on peut choisir un noyau.
3	**Initialiser le problème d'optimisation**	Construire l'objectif $\frac{1}{2}\|\|w\|\|^2 + C\Sigma\xi_i$, avec $\xi_i \geq 0$.
4	**Optimiser**	Un solveur cherche les valeurs de w, b et ξ_i qui minimisent l'objectif tout en respectant les contraintes.
5	**Construire la fonction de décision**	Calculer $f(x)=w^T x+b$ pour une nouvelle observation.
6	**Prédire la classe**	Si $f(x) \geq 0$, prédire +1 ; sinon, prédire −1.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
Naïve Bayes
.....

SVM

Exemple : trois observations

Pour comprendre ξ_i, supposons qu'un modèle ait déjà appris la frontière $f(x)=x-1$.

Classification binaire en une dimension : $y = +1$ ou $y = -1$; $f(x)=x-1$.

	x_i	y_i	$f(x_i)=x_i-1$	$z_i=y_i f(x_i)$
A	3	+1	2	2
B	1,4	+1	0,4	0,4
C	1,3	−1	0,3	−0,3

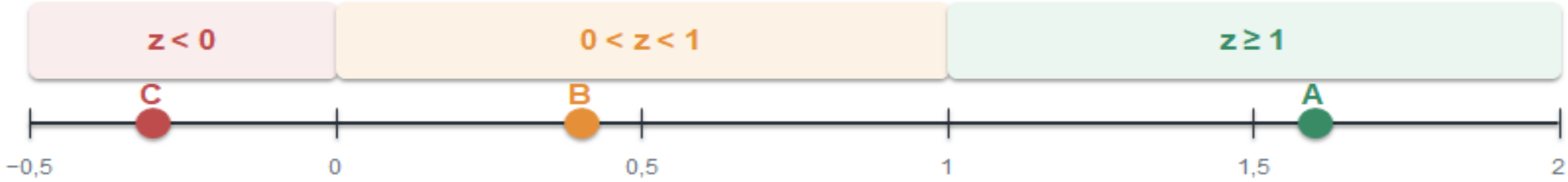
La frontière est $f(x)=0$, donc $x=1$. Les points A et B sont du côté positif ($f>0$). Le point C est du côté négatif ($f>0$ mais $y=-1$) : il est mal classé.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
Naïve Bayes
.....

SVM

Exemple : trois observations



Rappel : $\xi_i = \max(0, 1 - z_i)$, où $z_i = y_i f(x_i)$.

A	**z = 2**	**ξ = max(0, 1−2) = 0**	La marge est respectée.
B	**z = 0,4**	**ξ = max(0, 1−0,4) = 0,6**	Bien classé, mais dans la marge.
C	**z = −0,3**	**ξ = max(0, 1−(−0,3)) = 1,3**	Mal classé : violation importante.

Somme des écarts : $\Sigma\xi_i = 0 + 0,6 + 1,3 = 1,9$.

Chapitre 4: Classification

Perceptron simple (introduction aux réseaux)
Arbres de décision et forêts
k-plus proches voisins
Naïve Bayes
.....

SVM

Exemple : trois observations

La pénalité totale due aux violations vaut $C \times \Sigma\xi_i = C \times 1{,}9$.

C = 0,1	C = 1	C = 10
Pénalité = 0,19	**Pénalité = 1,9**	**Pénalité = 19**
Les violations coûtent peu dans l'objectif.	Les violations ont un coût intermédiaire.	Les violations sont fortement pénalisées.

C petit : le modèle accepte plus facilement des violations si cela permet de garder une marge plus large.
C grand : il cherche davantage à éviter les violations, parfois au prix d'une marge plus étroite.

C ne modifie pas directement ξ_i : il modifie le poids de leur somme dans l'optimisation.

Que ce soit le SVM à marge dure (Hard-Margin) ou le SVM à marge souple (Soft-Margin), l'apprentissage cherche principalement à optimiser les mêmes paramètres : w et b.
La différence est que le Soft-Margin introduit des paramètres supplémentaires : les variables d'écart ξ_i.

1. SVM à marge dure (Hard-Margin)

Le SVM cherche à déterminer :

- w : le vecteur des poids, qui détermine l'orientation de l'hyperplan.
- b : le biais, qui détermine la position de l'hyperplan.

Il résout le problème d'optimisation :

$$\boxed{\min_{w,b} \frac{1}{2}\|w\|^2}$$

Sous les contraintes :

$$y_i(w^T x_i + b) \geq 1$$

Il optimise donc w et b, sans variables d'écart.

2. SVM à marge souple (Soft-Margin)

Il cherche également à déterminer :

- (w) : $les\ poids$.
- (b) : $le\ biais$.
- (ξ_i): $les\ variables\ d'écart\ associées\ aux\ observations$.

Il résout :

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_i \xi_i$$

Sous les contraintes :

$$y_i(w^T x_i + b) \geq 1 - \xi_i$$

$$\xi_i \geq 0$$

Ici, C est un hyperparamètre choisi avant l'entraînement (généralement par validation croisée). Il n'est pas optimisé directement dans ce problème.